



Informe Tarea 1

Nombre: Felipe Robino Schütze

Pregunta 1

(a) El espacio de estados $\mathcal{S}$ se define como sigue:

$$\mathcal{S} := \{s_t = (B_t, D_t, L_t, G_t) | t \in \mathbb{Z}\}$$

con:

$$B_t \in \{0, 1, 2, 3, 4, 5\}$$

$$D_t \in \{m, a, n\}$$

$$L_t \in \{1, 2, 3\}$$

$$G_t \in \{0, 1, 2, 3\}$$

Luego, para cada t :

$$|s_t| = |B_t| \times |D_t| \times |L_t| \times |G_t| = 6 \times 3 \times 3 \times 4 = \boxed{216}$$

El espacio de acciones es:

$$\mathcal{A} := \{-2, -1, 0, 1\}$$

$$|\mathcal{A}| = 4$$

(b) La función de rewards diseñada se hizo con los siguientes criterios en mente:

- Reducir el uso de generación con Diesel
- Evitar brechas de suministro
- Limitar el ciclado de la batería
- Priorizar cargar la batería antes de la noche (reserva energética para la noche)
- No desperdiciar exedentes de generación

Formalmente:

$$r(s_t, a, s_{t+1}) = - \left(12d + |\tilde{a}| + 6 \cdot \mathbf{1}_{\{D=n\}} \cdot \mathbf{1}_{\{B \leq 1, d > 0\}} + 0.5 \cdot \text{spill} + 6|\tilde{a} - a| \right)$$

Donde:

$d_t = \max\{0, L_t - G_t + a\}$ – es el déficit cubierto con diesel.

$\tilde{a}$ – es la acción realmente tomada (e.g. no podemos cargar bat llena, etc.).

$\text{spill} = \max\{0, G_t + a - L_t\}$ – Es el exedente desperdiciado.

$\mathbf{1}_{\{B \leq 1, d > 0\}}$ – Es 1 cuando hay batería baja durante la noche.

(c) Si planteamos la ecuación de Bellman para nuestro sistema, tenemos:

$$v_\pi(s) = \sum_a \pi(a|s) \sum_{s',r} p(s'|s,a)[r + \gamma v_\pi(s')]$$

Asumiendo que los valores fueron inicializados en 0 ($v_\pi(s') = 0, \forall s'$), y que el controlador es determinístico ($\pi(a|s) = 1$ para $a = a_t$ y 0 para todo otro a), tenemos para el estado y acción del enunciado:

$$v_\pi(s) = v_\pi(s_t = (B_t = 2, D_t = m, L_t = 3, G_t = 2)) = \sum_{s_{t+1}, r} p(s_{t+1}|s_t, a_t = -1) \cdot r(s_t, a_t, s_{t+1})$$

Para encontrar la distribución de probabilidad del estado siguiente, usamos las matrices de transición que fueron entregadas en `p1.py`, por composición podemos encontrarlas como se aprecia en la Figura 1.

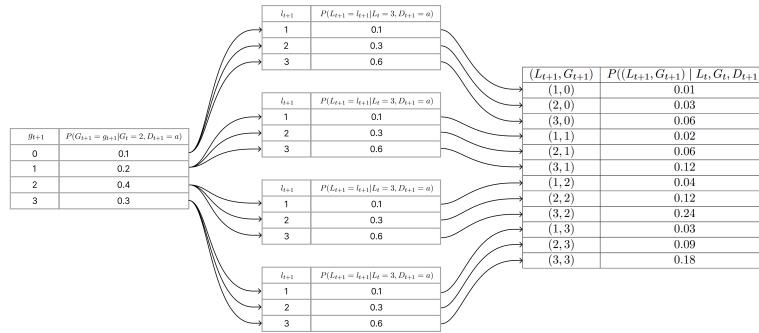


Figura 1: Probabilidades de estados futuros (L_{t+1}, G_{t+1}) para el estado del enunciado.

Así:

$$\begin{aligned} v_\pi(s_t) &= \sum_{s_{t+1}, r} p(s_{t+1}|s_t, a_t = -1) \cdot r(s_t, a_t, s_{t+1}) \\ &= 0.01 \cdot r(s_t, a_t = -1, s_{t+1} = (B_{t+1} = 1, D_{t+1} = a, L_{t+1} = 1, G_{t+1} = 0)) + \\ &\quad + 0.03 \cdot r(s_t, a_t = -1, s_{t+1} = (\cdot, \cdot, L_{t+1} = 2, G_{t+1} = 0)) \\ &\quad + 0.06 \cdot r(s_t, a_t = -1, s_{t+1} = (\cdot, \cdot, L_{t+1} = 3, G_{t+1} = 0)) \\ &\quad + 0.02 \cdot r(s_t, a_t = -1, s_{t+1} = (\cdot, \cdot, L_{t+1} = 1, G_{t+1} = 1)) \\ &\quad + 0.06 \cdot r(s_t, a_t = -1, s_{t+1} = (\cdot, \cdot, L_{t+1} = 2, G_{t+1} = 1)) \\ &\quad + 0.12 \cdot r(s_t, a_t = -1, s_{t+1} = (\cdot, \cdot, L_{t+1} = 3, G_{t+1} = 1)) \\ &\quad + 0.04 \cdot r(s_t, a_t = -1, s_{t+1} = (\cdot, \cdot, L_{t+1} = 1, G_{t+1} = 2)) \\ &\quad + 0.12 \cdot r(s_t, a_t = -1, s_{t+1} = (\cdot, \cdot, L_{t+1} = 2, G_{t+1} = 2)) \\ &\quad + 0.24 \cdot r(s_t, a_t = -1, s_{t+1} = (\cdot, \cdot, L_{t+1} = 3, G_{t+1} = 2)) \\ &\quad + 0.03 \cdot r(s_t, a_t = -1, s_{t+1} = (\cdot, \cdot, L_{t+1} = 1, G_{t+1} = 3)) \\ &\quad + 0.09 \cdot r(s_t, a_t = -1, s_{t+1} = (\cdot, \cdot, L_{t+1} = 2, G_{t+1} = 3)) \\ &\quad + 0.18 \cdot r(s_t, a_t = -1, s_{t+1} = (\cdot, \cdot, L_{t+1} = 3, G_{t+1} = 3)) \end{aligned}$$

Con la función de rewards diseñada, podemos evaluar los $r(\cdot)$. Sabemos que no se usa diesel, la batería se descarga en 1 ($|a| = 1$), no hay spill, y no hay acción inválida. Luego para todo estado futuro el reward será:

$$r(\cdot) = -(12 \cdot 0 + |1| + 6 \cdot 0 \cdot 0 + 0.5 \cdot \cdot 0 + 6|0|) = \boxed{-1}$$

Finalmente:

$$\begin{aligned} v_{\pi}(s_t) &= 0.01 \cdot -1 + 0.03 \cdot -1 + 0.06 \cdot -1 + 0.02 \cdot -1 \\ &\quad + 0.06 \cdot -1 + 0.12 \cdot -1 + 0.04 \cdot -1 + 0.12 \cdot -1 \\ &\quad + 0.24 \cdot -1 + 0.03 \cdot -1 + 0.09 \cdot -1 + 0.18 \cdot -1 \end{aligned}$$

$$\boxed{v_{\pi}(s_t) = -1}$$

- (d) Se implementaron ambos métodos utilizando de base el código dado en la clase 3. Ambos entrenamientos ocurren de manera consecutiva al correr `p1.py`, y ambos llegan a la misma política óptima y valores para seeds iguales. El número de iteraciones se imprime en la consola, y son los siguientes:

Policy Iteration:

Iteraciones externas: 5

Iteraciones internas totales (evaluación de política): 5374

Iteraciones internas por ciclo externo: [1089, 1078, 1069, 1069, 1069]

Iteraciones totales PI (internas + externas): 5379

Value Iteration:

Iteraciones: 1077 Como es de esperarse, VI requiere de significativamente menos iteraciones para converger. Esto se debe a que evalúa una sola vez antes de actualizar la política, mientras que PI realiza muchas evaluaciones de la política actual (iteraciones internas) antes de actualizarla. Mientras que PI necesita muchas menos actualizaciones de política para converger (5 en total - iteraciones externas), VI es mucho más eficiente computacionalmente, puesto que necesita de $\approx 80\%$ menos de iteraciones para llegar a exactamente la misma política, apreciable en la Figura 2.

- (e) Como se mencionó previamente, ambos algoritmos llegaron a la misma política óptima (Figura 2). Se puede apreciar claramente que para la función de rewards diseñada, el momento del día D_t no es relevante al momento de tomar decisiones, puesto que la acción es igual para todo momento (Los valores de cada estado no lo son, pero la acción óptima sí).

- Conviene cargar la batería ($a = 1$) cuando existe un excedente energético y la batería no está cargada completamente (Específicamente $G_t - L_t \geq 1$ y $B_t \leq 4$).
- Conviene descargarla cuando hay déficit de generación y la batería tiene carga significativa:
 - $G_t - L_t \leq -1$ y $B_t \geq 3 \rightarrow$ Descargamos $a_t = -1$
 - $G_t - L_t \leq -2$ y $B_t \geq 4 \rightarrow$ Descargamos fuertemente $a_t = -2$
- Si vemos la Figura 4 y Figura 3 (ambas trayectorias idénticas debido a mismo seed), podemos apreciar el comportamiento recién descrito del controlador. Se carga la batería cuando B_t no es máximo y existe un excedente de generación que lo permita. Se descarga para compensar déficits cuando la batería lo permite.

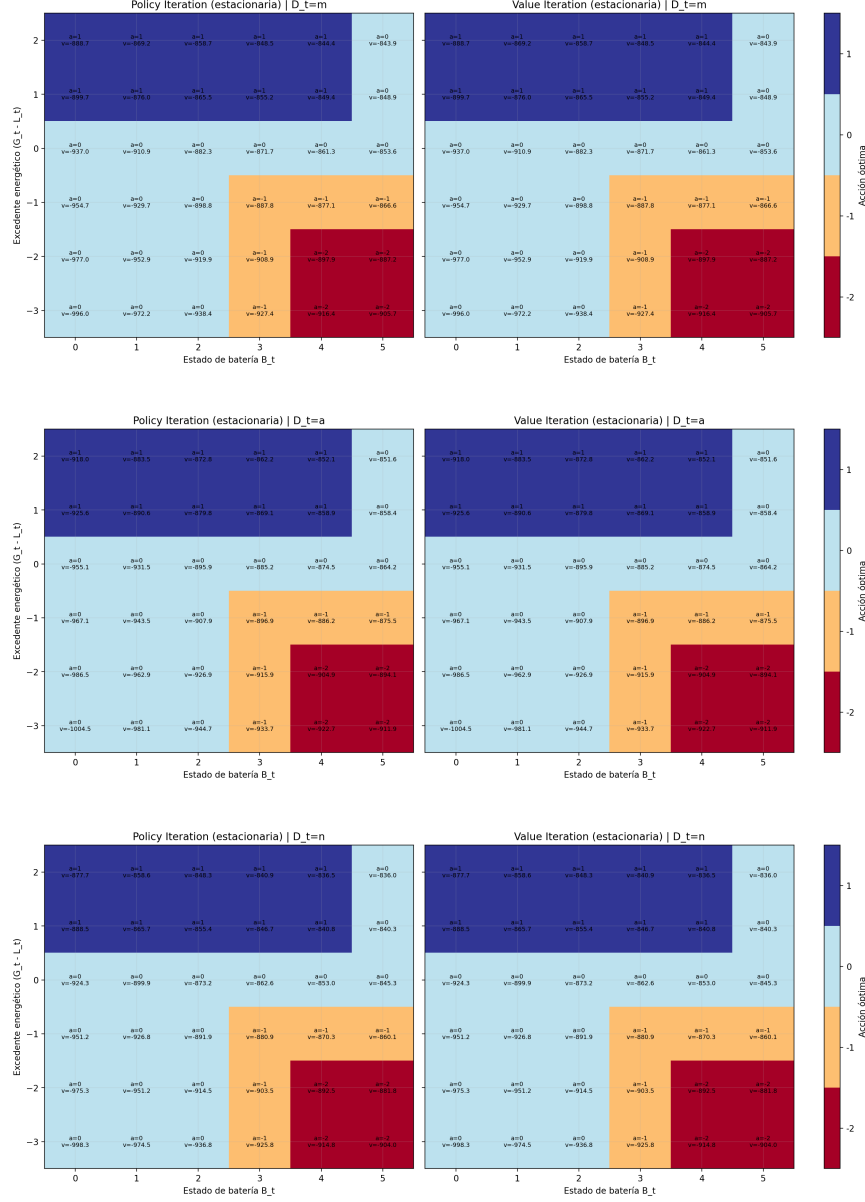


Figura 2: Política óptima encontrada para cada algoritmo (PI y VI), con la acción óptima y el valor asignado a cada estado respecto a una grilla $(G_t - L_t)$ vs B_t , para $D_t = m$, $D_t = a$ y $D_t = n$ (de arriba hacia abajo).

Pregunta 2

- (a) Al ejecutar el Código 0.0.2 p2.py, se pueden apreciar las diferencias entre el modelo no lineal, el linealizado, y la estimación de estado realizada por el filtro de Kalman. La respuesta del modelo linealizado difiere de la del no lineal al alejarse del punto de equilibrio sobre el cual fue linealizado. Esto es una regla general para todo modelo linealizado a partir de otro no lineal. Al alejarse del punto de equilibrio inicial, las aproximaciones iniciales que asimilaban los modelos dejan de ser válidas, y el modelo lineal deja de representar fielmente al no lineal.

Respecto a la estimación del KF, se puede apreciar que a grandes rasgos sigue de mucha mejor

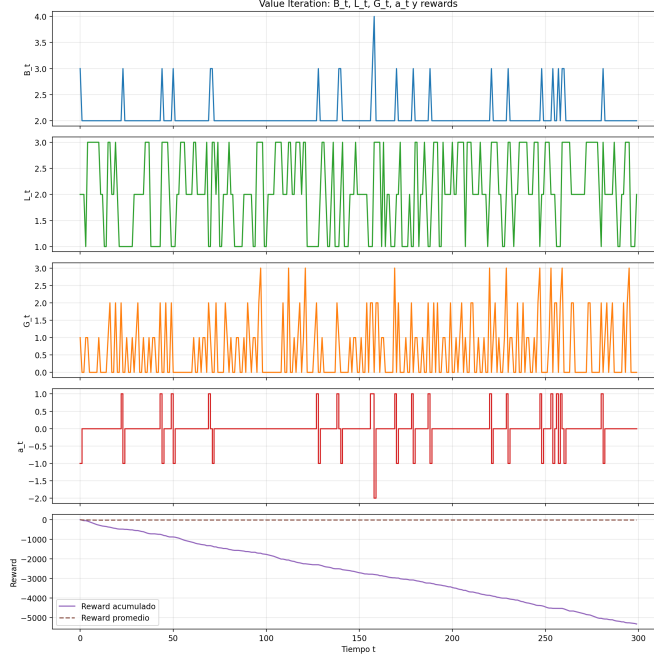


Figura 3: Trayectoria del controlador VI para 100 días de ciclado (300 t 's).

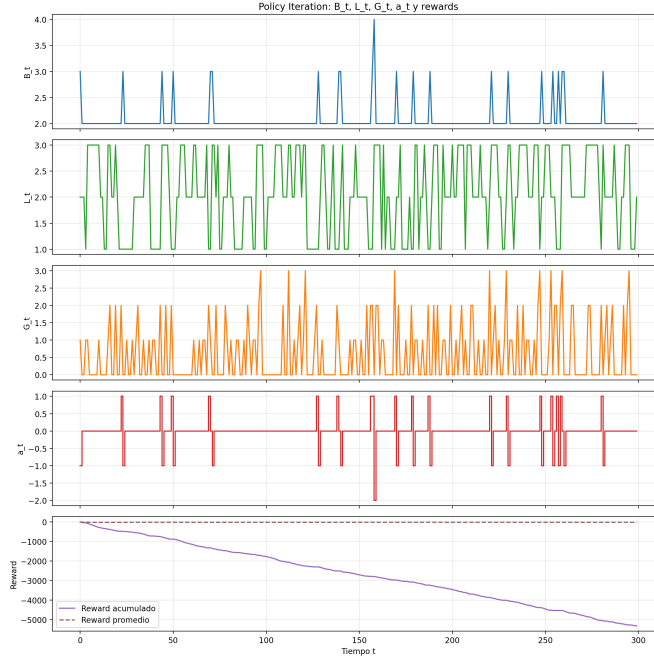


Figura 4: Trayectoria del controlador PI para 100 días de ciclado (300 t 's).

manera la curva del modelo no lineal. Esto se debe a que su arquitectura de predicción-corrección le permite ajustarse a medida que toma nuevas mediciones. Estas mediciones son ruidosas e imperfectas, y por eso la curva se ve mucho más ruidosa que la no lineal. Sin embargo, mediante las matrices R y Q , se puede compensar y ajustar qué tanto el filtro tiende a “creerle” a la medición o al modelo. El filtro de Kalman, es un estimador de estados lineales que asume ruido gaussiano en sus mediciones.

Funciona mediante dos pasos iterativos, la predicción seguida de una corrección asociada a una medición. La idea es que a partir de un modelo lineal de un sistema dinámico, el filtro realiza una predicción de un estado futuro y la covarianza del modelo. Luego, mide el estado real y lo compara con la predicción realizada, y ajusta la estimación mediante una ganancia de Kalman K que depende de Q y R . Su rol en sistemas de control es el de estimar estados que son medidos imperfectamente o que no son medibles (variables internas), filtrando el ruido de medición. Esto permite el control por realimentación de plantas no medibles completamente, y es la base de el LQG y el principal estimador de estado utilizado en general.

- (b) Se generó el Código 0.0.4 `p2_open_loop.py` que realiza la simulación en lazo abierto descrita. En la Figura 5 se puede apreciar el comportamiento altamente acoplado del sistema. Esto significa que un escalon en la entrada asociada a *pitch* afecta no sólo a esa variable, sino que también a *yaw* y viceversa. Este acoplamiento imposibilita el control del sistema mediante controladores PID independientes, puesto que controlar cada variable independientemente no es viable. Más en profundidad, el control mediante PIDs para cada variable asume una planta SISO, mientras que el comportamiento visto indica que es una planta con fuerte acoplamiento MIMO. Si se implementaran 2 PIDs, probablemente competirían entre sí debido al acoplamiento y se llegaría a oscilaciones o inestabilidades generadas por los mismos controladores.

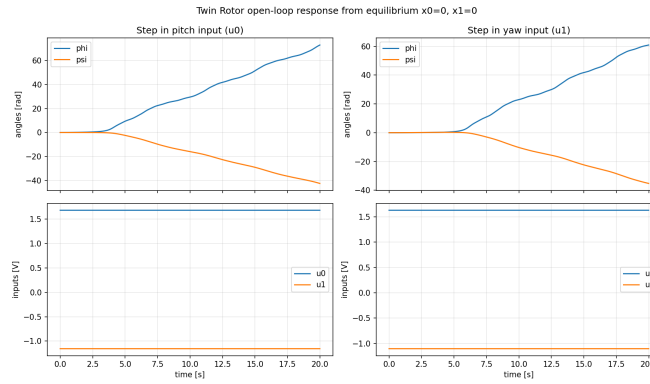


Figura 5: Respuesta del sistema en lazo abierto para un escalon en la entrada de *pitch* (izquierda) y *yaw* (derecha).

- (c) A partir de la ecuación de Bellman, podemos plantear la función de costo:

$$J = \sum_{k=0}^{N-1} (x_k^\top Q x_k + u_k^\top R u_k) + x_N^\top P_N x_N$$

Para un LQR, tomamos $V_k(x) = x^\top P_k x$. Reemplazando en J :

$$V_k(x) = \min_u (x^\top Q x + u^\top R u + (Ax + Bu)^\top P_{k+1} (Ax + Bu))$$

Si derivamos respecto a u e igualamos a 0:

$$u_k^* = -K_k x_k$$

Con:

$$K_k = (R + B^\top P_{k+1} B)^{-1} B^\top P_{k+1} A$$

La recursión de Ricatti para esto es:

$$P_k = Q + A^\top P_{k+1} A - A^\top P_{k+1} B (R + B^\top P_{k+1} B)^{-1} B^\top P_{k+1} A$$

Luego, podemos implementar esta recursión partiendo con un N grande y un P_N arbitrario ($P_N = Q$ en el código) e iterar hacia atrás hasta llegar al valor final. Este procedimiento fue implementado en las líneas 33 a 51 del Código 0.0.5 `p2_lqr_dp.py`, entregando los siguientes resultados:

```
Iterations until convergence: 4152
Final Riccati delta: 9.995e-08
K from dynamic programming:
[[ 5.4366109  11.86057678 52.20581777  6.01035488  8.79935688  0.23624256]
 [-0.69628894  4.57516771 -8.26397584 56.65467288  0.23957641  6.49529429]]

K from control.dlqr:
[[ 5.43932702 11.86063255 52.20590004  6.01033097  8.79936024  0.2362423 ]
 [-0.69660522  4.57517165 -8.26398555 56.65467641  0.23957602  6.49529433]]

||K_dp - K_dlqr||:
0.0027364081921502892
Relative difference:
3.4336705842740536e-05
```

Se puede apreciar la buena calidad de la convergencia de la programación dinámica, que llega con 4152 iteraciones a un valor muy similar al del obtenido mediante `control.dlqr`. Las ganancias obtenidas mediante ambos métodos llegan a una diferencia absoluta de 0.0027364 y relativa de 3.4336705e-5, valores muy pequeños y que validan la aproximación mediante Bellman y Ricatti para obtener el controlador LQR óptimo.

- (d) Se generó el Código 0.0.3 `p2_closed_loop.py`, que realiza el trabajo de simular las 3 referencias $((x_0, x_1) = \{(0, 0); (0, \frac{\pi}{4}); (\frac{\pi}{4}, \frac{\pi}{4})\})$ distintas, junto con la perturbación indicada en $t = [1000, 1500]$ con amplitud 0.1. La linealización del sistema respecto a los distintos puntos de equilibrio se hizo con la función `get_linear_system` de `TwinRotor`, y sobre esto se utilizó un KF y LQR para controlar la planta. Al correr el script se imprimen las métricas de IEA (*Integral Absolute Error*) para todos los casos. Se mide el IAE total (todo el horizonte de simulación), e IAE_dist que sólo considera la ventana de perturbación. Los resultados del rechazo a la perturbación se pueden apreciar en la Figura 6. El output de IEAs medidos es el siguiente:

```
Case 1 ref=(0.00000, 0.00000)
| IAE_phi=2.3454, IAE_psi=1.0538, IAE_total=3.3992, IAE_dist=3.3125
Case 2 ref=(0.00000, 0.78540)
| IAE_phi=2.1646, IAE_psi=1.1055, IAE_total=3.2700, IAE_dist=3.1767
Case 3 ref=(0.78540, 0.78540)
| IAE_phi=1.9688, IAE_psi=1.1452, IAE_total=3.1140, IAE_dist=3.0164
```

Se puede apreciar numéricamente como los IEAs se mantienen bastante similares entre todos los puntos de equilibrio. Esto da fé de cómo el lazo cerrado KF+LQR implementado es estable y rechaza perturbaciones bajo distintos puntos de equilibrio de manera correcta.

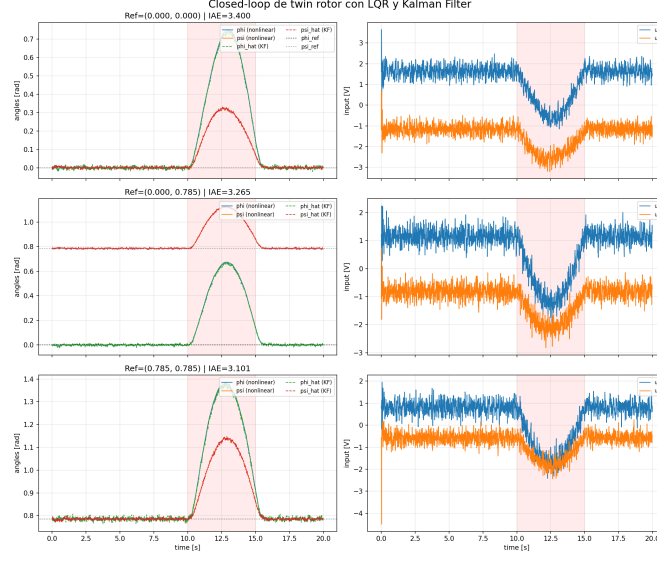


Figura 6: Gráficos de rechazo de perturbación para los distintos puntos de equilibrio planteados. En rojo la ventana de perturbación, Las mediciones del KF a la izquierda y la acción de control a la derecha.

- (e) Para realizar la simulación con a través de la red de comunicaciones, se generó el Código 0.0.6 `p2_NCS.py`, que realiza tanto la simulación con Thompson Sampling como la Epsilon-Greedy sobre la selección de canales. La simulación con Thompson Sampling se realiza asumiendo $\alpha = 1$ y $\beta = 1$ inicialmente para todos los canales, y a medida que se muestrea se generan las distribuciones reales siguiendo la lógica de: muestreo $\rightarrow$ selección de mejor canal $\rightarrow$ acción $\rightarrow$ actualización de distribuciones. Desde el lado epsilon-greedy, se utilizó $\varepsilon = 0.1$ constante como benchmark. Los resultados de la selección de canales de ambas estrategias y sus rewards se pueden apreciar en la Figura 7. Los posteriors generados por Thompson Sampling se pueden apreciar en la ??, siendo Ch1 el más óptimo. La comparación del desempeño entre ambas estrategias se puede apreciar analizando el regret acumulado de cada estrategia ($1 - r$ en este caso), que se ve en la Figura 9, donde se visualiza claramente la mejor calidad de exploración que entrega Thompson Sampling por sobre Epsilon-Greedy (menor regret). Pese a la mayor complejidad, Thompson Sampling es una mejor estrategia de exploración a largo plazo que Epsilon-Greedy, ya que prioriza muestrear aquellas acciones de interés y no se mantiene explorando acciones irrelevantes (con muy mal reward esperado) como es el caso de su contraparte.

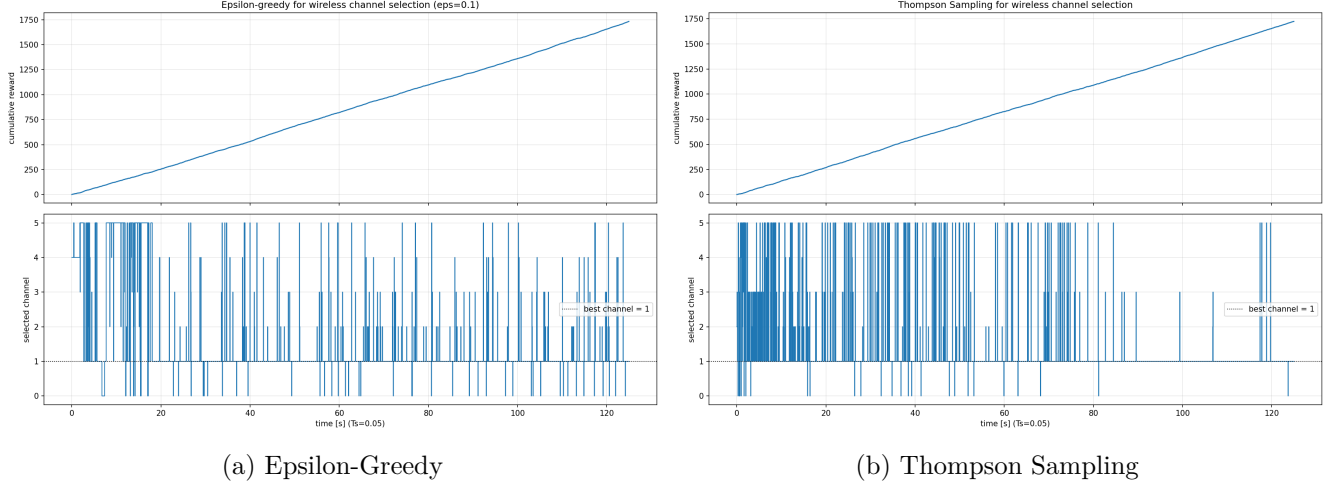


Figura 7: Selección de canales por paso y reward acumulado de ambas estrategias de exploración para el NCS.

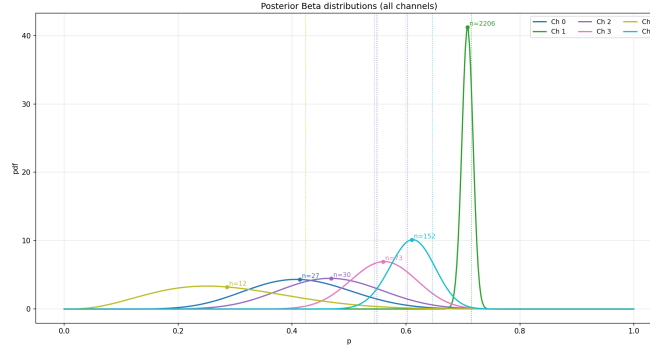


Figura 8: Distribuciones beta generadas por Thompson Sampling a partir del muestreo.

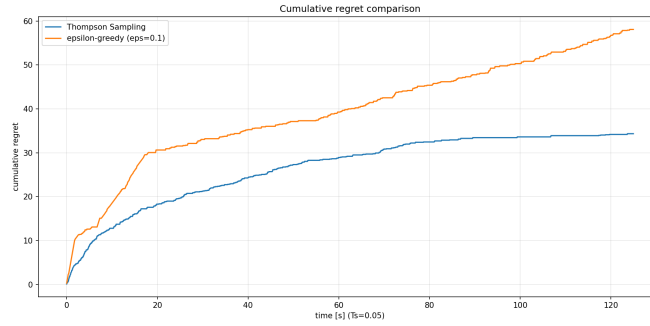


Figura 9: Regret acumulado de ambas estrategias de exploración.

Pregunta 3

- (a) El Multi-Agent Reinforcement Learning (MARL) es una estrategia de RL que tiene por principio la participación de múltiples agenets que aprenden simultáneamente mediante interacciones con un entorno común y entre ellos. La idea es que cada agente sigue su propia política, y las acciones tomadas por dicho agente afectan el entorno y por ende el aprendizaje de los otros agentes. Esto

sirve para modelar comportamientos más complejos de cooperación o competencia. Existen distintos enfoques para llevar a cabo este entrenamiento, entre los cuales están:

- *Centralized*: Existe un controlador central que observa el estado global y decide acciones para todos los agentes (i.e. cada agente es parte de un estado global que se controla desde un controlador). Esto tiene la ventaja de ser más fácil de interpretar, y que tiene visibilidad completa de información y puede llevar a mejor coordinación entre los agentes. Sin embargo, asume que existe esta forma centralizada de ver los datos, que muchas veces no es normal para escenarios donde los agentes tienen poca o nula visibilidad entre ellos.
- *Decentralized*: Cada agente toma sus propias decisiones de forma completamente autónoma. Esto se puede combinar con un entrenamiento centralizado pero ejecución descentralizada, que permite buscar comportamientos no descubribles desde el estado de un solo agente. Tiene la ventaja de ser más realista y considerar los estados visibles desde cada agente, pero complejiza la coordinación y el acceso a la información de cada agente.
- *Independent*: Este enfoque hace que los agentes aprendan como si fueran los únicos en el entorno, es decir, trata a todo otro agente como parte del entorno. Esto tiene la ventaja de ser simple de implementar y realista en el enfoque de acceso a la información, pero puede obviar comportamientos de los otros agentes y por ende llevar a peores desempeños en tareas que necesitan cooperación.

Para problemas como el del enunciado, donde hay múltiples entidades o agentes que actúan de manera individual, es la manera más natural de modelarlo. Cada persona en el edificio no toma decisiones basándose en el estado global del edificio, sino que toma la información a la que tiene acceso (su entorno inmediato y otros agentes) y decide acciones basándose en esto, que a su vez afectan al entorno y a los otros agentes. Así, la mejor forma de modelar y escalar este comportamiento es mediante MARL. La escalabilidad de estos modelos también es mejor debido a que el espacio de acciones y estados por agente es más o menos constante respecto al número de agentes, mientras que un controlador centralizado crece linealmente con el número de agentes. Este tipo de RL es muy útil para permitir que surjan naturalmente estrategias coordinadas y eficientes sin programarlas explícitamente, ya que en teoría con una buena función de rewards bien diseñada los agentes debiesen llegar a comportamientos deseables de manera independiente y paralela, que serían obviados por un sólo agente.

- (b) Se generó el Código 0.0.7 p3.py a partir del código entregado, el cual realiza la tarea de entrenar dos environments del grafo y entrenar 50 agentes durante 1000 episodios. Se realiza SARSA y Q-Learning, con la diferencia clave en el enfoque multi agente siendo que en vez de una acción, se obtiene un array de acciones independientes para cada agente (`actions = [epsilon_greedy_policy(...)` `for ... in ...]`). Esto tiene el efecto de entrenar agentes de manera independiente y es la implementación más directa y fácil de interpretar de MARL que se pudo idear para el problema. Las implementaciones de SARSA y Q-Learning difieren en el arg máx que Q-learning utiliza para actualizar los valores de la q-table. Ambos utilizan política ϵ -greedy con ϵ -decay de 0.2 a 0.1 con 0.995 de decay-rate. Se generaron funciones helper para simular ambos ambientes, además de una función `plot_trajectories(...)` que ayuda a visualizar trayectorias de los agentes en el grafo.

Las curvas de entrenamiento por épocas de ambos algoritmos se pueden apreciar en la Figura 10. Se puede apreciar cómo Q-Learning logra alcanzar políticas mejores que SARSA, y bajo la política óptima encontrada logra salvar a 25 personas versus las 15 de SARSA. Esto se debe probablemente a que Q-Learning es off-policy, y debido a esto, es capaz de realizar un poco más de exploración que SARSA bajo los mismos parámetros de ϵ -decay que se tomaron. El aprendizaje off-policy permite

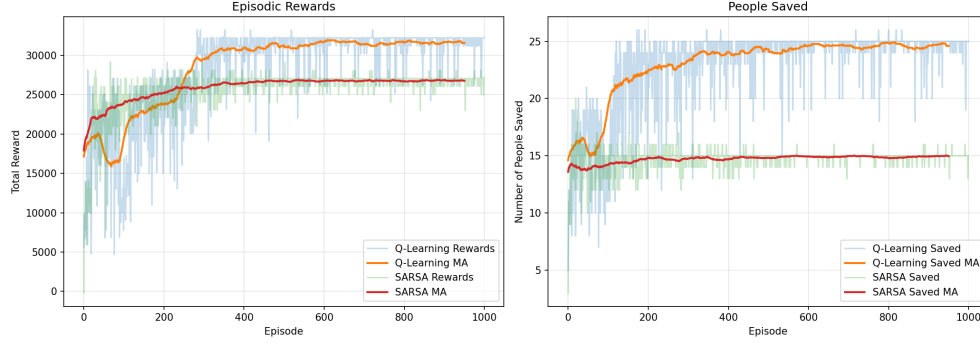


Figura 10: Curvas de entrenamiento de MARL con Q-Learning y SARSA para los 50 agentes y la configuración inicial del grafo. A la izquierda el reward acumulado por episodio (MA = moving average con ventana de 50 episodios), y a la derecha la cantidad de personas salvadas por episodio.

que los agentes realicen acciones distintas a las que se aprenden directamente en la tabla, y es este desacoplamiento entre política aprendida y ejecutada que puede dejar que se exploren políticas más diversas y posiblemente mejores que las que alcanza a explorar SARSA.

- (c) Como se mencionó anteriormente, se generó una función helper para visualizar de mejor manera las trayectorias de los agentes. Con esto, en la Figura 11 se puede apreciar el comportamiento aprendido de algunos de los agentes. Se puede ver cómo la regla general aprendida es que se dirijan a la salida más cercana, y cómo aquellos que no logran llegar en general no lo hacen porque las salidas les quedan demasiado lejos (línea café y verde agentes 30 y 48 de la figura). Esto es indicativo que no necesariamente la política aprendida es mala, si no que los parámetros (i.e. colocación de las salidas) del problema no dan abasto para un mejor desempeño en los 300s de simulación. El gráfico de personas salvadas en el tiempo se puede apreciar en la Figura 12.

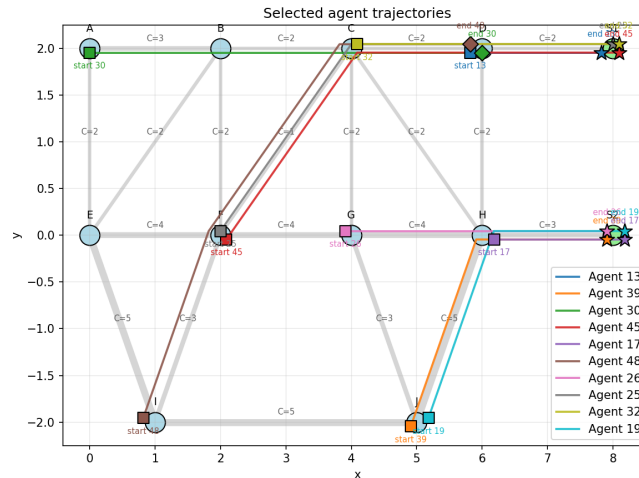


Figura 11: Trayectoria de algunos de los agentes entrenados para el caso base.

- (d) Para realizar la evaluación de las distintas configuraciones de salidas, se generó el Código 0.0.9 `p3_salidas.py`, que itera a través de todas las configuraciones posibles de salidas (asumiendo que S1 y S2 deben unirse a nodos distintos, con los mismos parámetros de capacidad y largo de antes) y obtiene las mejores basándose en el mejor rendimiento en número de personas salvadas para la

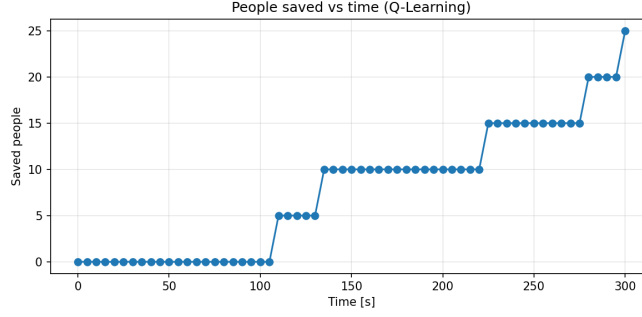


Figura 12: Personas salvadas con el tiempo para la política aprendida por Q-Learning.

última época de entrenamiento de Q-Learning. Las trayectorias pueden apreciarse en la Figura 14. Analizando la topología de los grafos de estas mejores configuraciones y contrastándolas contra el caso base, queda en evidencia algunas de las falencias de la configuración base. En general las soluciones buenas tienen a las salidas ancladas a puntos distantes entre sí. Esto permite que ninguna persona quede demasiado lejos de las salidas y en general logra encontrar alguna de las dos salidas alcanzable, idea que se refuerza al ver que todos los agentes se dirigen a la más cercana y no hay cruces (que un agente fuera a la salida lejana por congestión o alguna otra razón). Además, las salidas bien configuradas tienden a anclarse a nodos con muchos vecinos (i.e. accesibles) para facilitar que la concurrencia sea desde varias trayectorias (y mejorar la congestión así también). Esto se puede ver en que los nodos de 2 vecinos A y D no se usan para salida en ninguna de las configuraciones con mejor desempeño, y los con 3 I y J rara vez, que de todas maneras son viables debido a que tienen mucha capacidad por arista y por lo tanto mucha capacidad total pese a tener menos vecinos. Los mejores nodos para las salidas en general son más bien centrales y bien conectados, y una salida tiende a complementar a la otra colocándose en otro nodo suficientemente conectado pero también suficientemente lejano como para evitar comportamientos como el del caso base.

En la Figura 13 se puede apreciar cómo las personas salvadas durante los 300s de evacuación para las configuraciones mejor hechas son muy superiores que las de la configuración base. Las mejores 5 configuraciones llegan a salvar a 45 personas (90 % del total) bajo políticas entrenadas con el mismo algoritmo. Esto refuerza la idea de que la política aprendida es correcta, y que el mal desempeño del caso base se debe principalmente a la mala topografía y limitaciones propias del sistema más que de la política aprendida por cada agente.

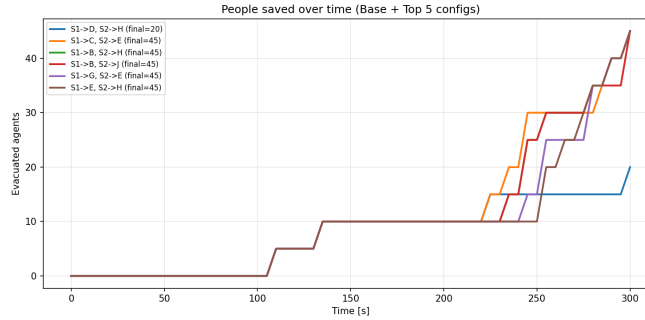


Figura 13: Personas salvadas respecto al tiempo para las 5 mejores configuraciones y el caso base.

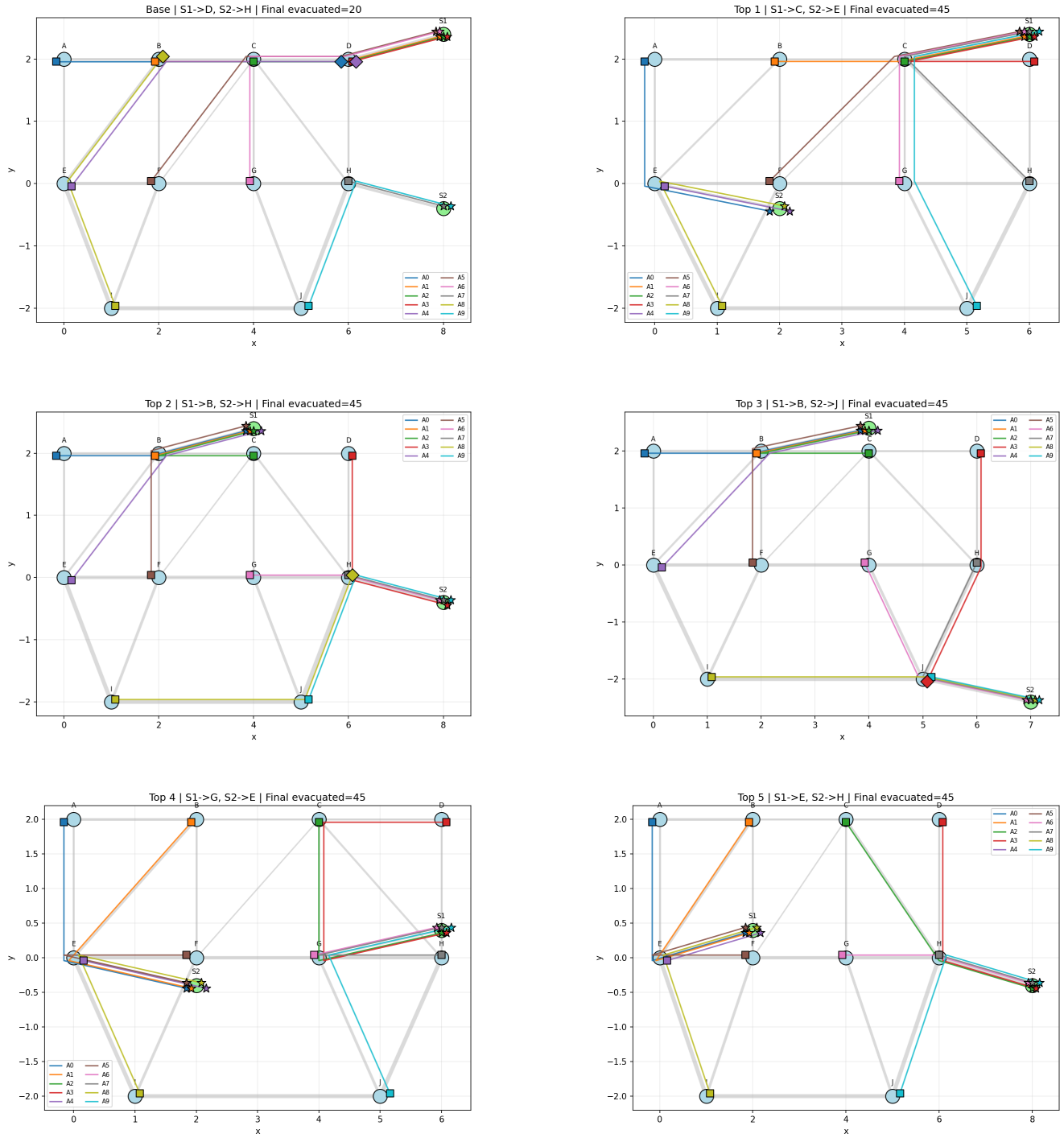


Figura 14: Trayectorias de agentes entrenados con Q-learning en las 5 mejores configuraciones y el caso base.

Anexos

Anexos Pregunta 1

0.0.1. p1.py

```

1
2 # La llave principal (m,a, n) es el momento del d a . La llave secundaria (1,2,3) es el
   valor de la variable en el tiempo t
3 # El diccionario interior son los valores en t+1 con su respectiva probabilidad
4 # 'm': {1: {1: 0.7, 2: 0.2, 3: 0.1} -> En este caso: Si es de ma ana y el valor en t es
   1, hay un 70% de probabilidad de permanecer en 1, 20% de transicionar a 2 y 10% de
   transicionar a 3
5 import numpy as np
6 import matplotlib.pyplot as plt
7 import os
8
9 L_trans_probs = {
10     'm': {
11         1: {1: 0.7, 2: 0.2, 3: 0.1},
12         2: {1: 0.2, 2: 0.6, 3: 0.2},
13         3: {1: 0.1, 2: 0.3, 3: 0.6},
14     },
15     'a': {
16         1: {1: 0.6, 2: 0.3, 3: 0.1},
17         2: {1: 0.2, 2: 0.5, 3: 0.3},
18         3: {1: 0.1, 2: 0.3, 3: 0.6},
19     },
20     'n': {
21         1: {1: 0.6, 2: 0.3, 3: 0.1},
22         2: {1: 0.2, 2: 0.5, 3: 0.3},
23         3: {1: 0.1, 2: 0.2, 3: 0.7},
24     }
25 }
26
27 G_trans_probs = {
28     'm': {
29         0: {0: 0.6, 1: 0.3, 2: 0.1, 3: 0.0},
30         1: {0: 0.2, 1: 0.5, 2: 0.3, 3: 0.0},
31         2: {0: 0.1, 1: 0.3, 2: 0.6, 3: 0.0},
32         3: {0: 0.1, 1: 0.2, 2: 0.5, 3: 0.2},
33     },
34     'a': {
35         0: {0: 0.4, 1: 0.3, 2: 0.2, 3: 0.1},
36         1: {0: 0.2, 1: 0.4, 2: 0.3, 3: 0.1},
37         2: {0: 0.1, 1: 0.2, 2: 0.4, 3: 0.3},
38         3: {0: 0.05, 1: 0.15, 2: 0.3, 3: 0.5},
39     },
40     'n': {
41         0: {0: 1.0, 1: 0.0, 2: 0.0, 3: 0.0},
42         1: {0: 1.0, 1: 0.0, 2: 0.0, 3: 0.0},
43         2: {0: 1.0, 1: 0.0, 2: 0.0, 3: 0.0},
44         3: {0: 1.0, 1: 0.0, 2: 0.0, 3: 0.0},
45     }
46 }
47
48 # POLICY ITERATION clase 3
49 def policy_evaluation(pi, P, gamma=1.0, theta=1e-10, max_iterations=10000):
50     V = np.zeros(len(P), dtype=np.float64)
51     inner_iterations = 0
52
53     for _ in range(max_iterations):
54         inner_iterations += 1
55         prev_V = V.copy()

```

```

56     V_new = np.zeros(len(P), dtype=np.float64)
57
58     for s in range(len(P)):
59         for prob, next_state, reward, done in P[s][pi[s]]:
60             continuation = 0.0 if done else gamma * V[next_state]
61             V_new[s] += prob * (reward + continuation)
62
63     V = V_new
64
65     if np.max(np.abs(V - prev_V)) < theta:
66         break
67
68     return V, inner_iterations
69
70 def policy_improvement(V, P, gamma=1.0):
71     Q = np.zeros((len(P), len(P[0])), dtype=np.float64)
72     for s in range(len(P)):
73         for a in range(len(P[s])):
74             for prob, next_state, reward, done in P[s][a]:
75                 Q[s][a] += prob * (reward + gamma * V[next_state] * (not done))
76     new_pi = {s:a for s, a in enumerate(np.argmax(Q, axis=1))}
77     return new_pi
78
79 def policy_iteration(P, gamma=1.0, theta=1e-10, max_outer_iterations=1000,
80 max_eval_iterations=10000):
81     random_actions = np.random.choice(tuple(P[0].keys()), len(P))
82     pi = {s: a for s, a in enumerate(random_actions)}
83     outer_iterations = 0
84     inner_iterations_total = 0
85     inner_iterations_per_outer = []
86
87     for _ in range(max_outer_iterations):
88         outer_iterations += 1
89         old_pi = {s:pi[s] for s in range(len(P))}
90         V, inner_iters = policy_evaluation(pi, P, gamma, theta, max_eval_iterations)
91         inner_iterations_total += inner_iters
92         inner_iterations_per_outer.append(inner_iters)
93         pi = policy_improvement(V, P, gamma)
94         if old_pi == {s:pi[s] for s in range(len(P))}:
95             break
96     return V, pi, outer_iterations, inner_iterations_total, inner_iterations_per_outer
97
98 # VALUE ITERATION
99 def value_iteration(P, gamma=0.98, theta=1e-10, max_iterations=10000):
100     V = np.zeros(len(P), dtype=np.float64)
101     iterations = 0
102
103     for _ in range(max_iterations):
104         iterations += 1
105         Q = np.zeros((len(P), len(P[0])), dtype=np.float64)
106
107         for s in range(len(P)):
108             for a in range(len(P[s])):
109                 for prob, next_state, reward, done in P[s][a]:
110                     continuation = 0.0 if done else gamma * V[next_state]
111                     Q[s][a] += prob * (reward + continuation)
112
113         V_new = np.max(Q, axis=1)
114         if np.max(np.abs(V_new - V)) < theta:

```

```

114         V = V_new
115         break
116     V = V_new
117
118     pi = {s: a for s, a in enumerate(np.argmax(Q, axis=1))}
119     return V, pi, iterations
120
121 #####
122 # MDP
123 # bateria
124 def next_battery(B, action):
125     return max(0, min(5, B + action))
126
127
128 def apply_control(B, L, G, action):
129     if action < 0:
130         discharge = min(-action, B)
131         effective_action = -discharge
132     elif action > 0:
133         solar_surplus = max(0, G - L)
134         charge = min(action, 5 - B, solar_surplus)
135         effective_action = charge
136     else:
137         effective_action = 0
138
139     B_next = next_battery(B, effective_action)
140
141     discharge = max(0, -effective_action)
142     charge = max(0, effective_action)
143     net_supply_for_load = G + discharge - charge
144
145     diesel = max(0, L - net_supply_for_load)
146     spill = max(0, net_supply_for_load - L)
147     invalid_magnitude = abs(action - effective_action)
148
149     return B_next, effective_action, diesel, spill, invalid_magnitude
150
151 # d a
152 def nex_d(D):
153     if D == 'm':
154         return 'a'
155     elif D == 'a':
156         return 'n'
157     else:
158         return 'm'
159
160 # defs espacios de acciones y estados
161 L_states = [1, 2, 3] # demanda
162 G_states = [0, 1, 2, 3] # generacion solar
163 D_states = ['m', 'a', 'n'] # momento del d a
164 actions = [-2, -1, 0, 1] # [-2 descargar bater a fuerte, -1 descargar bater a leve, 0
    mantener estado, 1 cargar bater a leve]
165 B_states = [0, 1, 2, 3, 4, 5] # carga bat
166
167 T = 300 # horizonte
168
169 # fun de reward
170 def reward(D, L, G, B, action):

```



```

171     _, effective_action, diesel, spill, invalid_magnitude = apply_control(B, L, G,
172                                     action)
173
174     # minimizar diesel, evitar deficit y no abusar de bater a.
175     c_diesel = 12.0 * diesel
176     c_battery = 1.0 * abs(effective_action) # proporcional al uso de bater a
177     c_invalid = 6.0 * invalid_magnitude
178     c_night_reserve = 6.0 if (D == 'n' and B <= 1 and diesel > 0) else 0.0 # reserva de
179                                     noche para prever deficit de gen solar
180     c_spill = 0.5 * spill # desperdiciar excedente
181
182     return -(c_diesel + c_battery + c_invalid + c_night_reserve + c_spill)
183 #####
184 # estados
185 states = []
186 states_to_index = {}
187
188 idx = 0
189 for B in B_states:
190     for D in D_states:
191         for L in L_states:
192             for G in G_states:
193                 s = (B, D, L, G)
194                 states.append(s)
195                 states_to_index[s] = idx
196                 idx += 1
197
198 #####
199 # P[s][a] = [(prob, next_state, reward, done), ...]
200 P = {s_idx: {a_idx: [] for a_idx in range(len(actions))} for s_idx in range(len(states))
201     }
202
203 for s_idx, (B, D, L, G) in enumerate(states):
204
205     for a_idx, a in enumerate(actions):
206         B_next, _, _, _ = apply_control(B, L, G, a)
207         D_next = nex_d(D)
208
209         # P(X_{t+1} | X_t, D_{t+1}).
210         for L_next, prob_L in L_trans_probs[D_next][L].items():
211             for G_next, prob_G in G_trans_probs[D_next][G].items():
212
213                 prob = prob_L * prob_G
214
215                 s_next = (B_next, D_next, L_next, G_next)
216                 s_next_idx = states_to_index[s_next]
217
218                 r = reward(D, L, G, B, a)
219
220                 done = False # no hay estado terminal
221
222                 P[s_idx][a_idx].append((prob, s_next_idx, r, done))
223
224 def _sample_from_probs(prob_dict, rng):
225     values = list(prob_dict.keys())
226     probs = list(prob_dict.values())
227     return rng.choice(values, p=probs)

```

```

227 def simulate_policy_trajectory(pi, T, initial_state, seed=123):
228     rng = np.random.default_rng(seed)
229
230     B_hist = np.zeros(T, dtype=np.int32)
231     L_hist = np.zeros(T, dtype=np.int32)
232     G_hist = np.zeros(T, dtype=np.int32)
233     a_hist = np.zeros(T, dtype=np.int32)
234     r_hist = np.zeros(T, dtype=np.float64)
235     diesel_hist = np.zeros(T, dtype=np.float64)
236
237     state = initial_state
238
239     for t in range(T):
240         B, D, L, G = state
241         s_idx = states_to_index[state]
242
243         if isinstance(pi, list):
244             # Pol tica dependiente del tiempo (ej. value iteration finito).
245             a_idx = pi[min(t, len(pi) - 1)][s_idx]
246         else:
247             # Pol tica estacionaria (ej. policy iteration).
248             a_idx = pi[s_idx]
249
250         a = actions[a_idx]
251         B_next, a_effective, diesel, _, _ = apply_control(B, L, G, a)
252
253         B_hist[t] = B
254         L_hist[t] = L
255         G_hist[t] = G
256         a_hist[t] = a_effective
257         r_hist[t] = reward(D, L, G, B, a)
258         diesel_hist[t] = diesel
259
260         D_next = nex_d(D)
261         #  $P(X_{t+1} \mid X_t, D_{t+1})$ .
262         L_next = _sample_from_probs(L_trans_probs[D_next][L], rng)
263         G_next = _sample_from_probs(G_trans_probs[D_next][G], rng)
264         state = (B_next, D_next, L_next, G_next)
265
266     R_cum = np.cumsum(r_hist)
267     R_avg = R_cum / (np.arange(T) + 1)
268     return B_hist, L_hist, G_hist, a_hist, R_cum, R_avg, r_hist, diesel_hist
269
270
271 def plot_policy_trajectory(title, B_hist, L_hist, G_hist, a_hist, R_cum, R_avg,
272                             save_path=None):
273     t = np.arange(len(B_hist))
274     fig, axes = plt.subplots(5, 1, figsize=(12, 12), sharex=True)
275
276     axes[0].plot(t, B_hist, color='tab:blue')
277     axes[0].set_ylabel('B_t')
278     axes[0].set_title(title)
279     axes[0].grid(alpha=0.3)
280
281     axes[1].plot(t, L_hist, color='tab:green')
282     axes[1].set_ylabel('L_t')
283     axes[1].grid(alpha=0.3)
284
285     axes[2].plot(t, G_hist, color='tab:orange')

```

```

285 axes[2].set_ylabel('G_t')
286 axes[2].grid(alpha=0.3)
287
288 axes[3].step(t, a_hist, where='post', color='tab:red')
289 axes[3].set_ylabel('a_t')
290 axes[3].grid(alpha=0.3)
291
292 axes[4].plot(t, R_cum, color='tab:purple', label='Reward_acumulado')
293 axes[4].plot(t, R_avg, color='tab:brown', linestyle='--', label='Reward_promedio')
294 axes[4].set_ylabel('Reward')
295 axes[4].set_xlabel('Tiempo_t')
296 axes[4].legend()
297 axes[4].grid(alpha=0.3)
298
299 plt.tight_layout()
300 if save_path is not None:
301     plt.savefig(save_path, dpi=200, bbox_inches='tight')
302 plt.close(fig)
303
304
305 def build_action_grid_by_battery_and_surplus(pi, V, D_fixed, t_policy=0):
306     surplus_values = np.arange(min(G_states) - max(L_states), max(G_states) - min(
307         L_states) + 1)
308     grid = np.full((len(surplus_values), len(B_states)), np.nan)
309     value_grid = np.full((len(surplus_values), len(B_states)), np.nan)
310
311     for b_idx, B in enumerate(B_states):
312         for s_idx, surplus in enumerate(surplus_values):
313             action_votes = []
314             state_values = []
315
316             for L in L_states:
317                 for G in G_states:
318                     if G - L != surplus:
319                         continue
320
321                     state = (B, D_fixed, L, G)
322                     state_idx = states_to_index[state]
323
324                     if isinstance(pi, list):
325                         a_idx = pi[min(t_policy, len(pi) - 1)][state_idx]
326                     else:
327                         a_idx = pi[state_idx]
328
329                     action_votes.append(int(a_idx))
330                     state_values.append(V[state_idx])
331
332             if action_votes:
333                 counts = np.bincount(action_votes, minlength=len(actions))
334                 grid[s_idx, b_idx] = float(np.argmax(counts))
335                 value_grid[s_idx, b_idx] = np.mean(state_values)
336
337     return grid, value_grid, surplus_values
338
339 def plot_policy_action_heatmaps_by_day(pi_left, pi_right, V_left, V_right, title_left,
340 title_right, t_right=0, save_dir=None):
341     cmap = plt.get_cmap('RdYlBu', len(actions))

```

```

342     for D_fixed in D_states:
343         grid_left, value_grid_left, surplus_values =
344             build_action_grid_by_battery_and_surplus(
345                 pi_left,
346                 V_left,
347                 D_fixed=D_fixed,
348                 t_policy=0
349             )
350         grid_right, value_grid_right, _ = build_action_grid_by_battery_and_surplus(
351             pi_right,
352             V_right,
353             D_fixed=D_fixed,
354             t_policy=t_right
355         )
356
357     fig, axes = plt.subplots(1, 2, figsize=(14, 6), sharey=True, constrained_layout=
358         True)
359     for ax, grid, value_grid, title in [(axes[0], grid_left, value_grid_left,
360         title_left), (axes[1], grid_right, value_grid_right, title_right)]:
361         im = ax.imshow(
362             grid,
363             origin='lower',
364             aspect='auto',
365             cmap=cmap,
366             vmin=-0.5,
367             vmax=len(actions) - 0.5,
368             extent=[B_states[0] - 0.5, B_states[-1] + 0.5, surplus_values[0] - 0.5,
369                 surplus_values[-1] + 0.5]
370         )
371
372         ax.set_title(f"{title}_D_t={D_fixed}")
373         ax.set_xlabel('Estado de bater a B_t')
374         ax.set_xticks(B_states)
375         ax.set_yticks(surplus_values)
376         ax.grid(alpha=0.2)
377
378         for yi, surplus in enumerate(surplus_values):
379             for xi, B in enumerate(B_states):
380                 if np.isnan(grid[yi, xi]):
381                     continue
382                 action_idx = int(grid[yi, xi])
383                 value = value_grid[yi, xi]
384                 text_str = f"a={actions[action_idx]}\nv={value:.1f}"
385                 ax.text(B, surplus, text_str, ha='center', va='center', color='black',
386                     , fontsize=7)
387
388     axes[0].set_ylabel('Excedente energ tico (G_t-L_t)')
389
390     cbar = fig.colorbar(im, ax=axes, fraction=0.046, pad=0.04)
391     cbar.set_label('Acci n ptima ')
392     cbar.set_ticks(np.arange(len(actions)))
393     cbar.set_ticklabels([str(a) for a in actions])
394
395     if save_dir is not None:
396         plt.savefig(os.path.join(save_dir, f'policy_comparison_heatmap_D_{D_fixed}.
397             png'), dpi=200, bbox_inches='tight')
398     plt.close(fig)

```

```

395 IMG_DIR = os.path.join(os.path.dirname(__file__), 'img')
396 os.makedirs(IMG_DIR, exist_ok=True)
397
398 GAMMA = 0.98
399 THETA = 1e-8
400 MAX_ITER = 10000
401
402 # Ejecutar policy/value iteration para el mismo problema estacionario descontado
403 V_pi, pi_pi, pi_outer, pi_inner_total, pi_inner_by_outer = policy_iteration(
404     P,
405     gamma=GAMMA,
406     theta=THETA,
407     max_outer_iterations=MAX_ITER,
408     max_eval_iterations=MAX_ITER
409 )
410 V_vi, pi_vi, iters_vi = value_iteration(P, gamma=GAMMA, theta=THETA, max_iterations=
    MAX_ITER)
411
412 print("Policy Iteration:")
413 print("Valor de los estados:", V_pi)
414 print("Iteraciones externas:", pi_outer)
415 print("Iteraciones internas totales (evaluación de política):", pi_inner_total)
416 print("Iteraciones internas por ciclo externo:", pi_inner_by_outer)
417 print("Iteraciones totales PI (internas + externas):", pi_outer + pi_inner_total)
418
419 print("\nValue Iteration:")
420 print("Valor de los estados:", V_vi)
421 print("Iteraciones:", iters_vi)
422
423 # Estado inicial
424 initial_state = (3, 'm', 2, 1)
425
426 # Gráfico Policy Iteration
427 B_pi, L_pi, G_pi, a_pi, Rcum_pi, Ravg_pi, r_pi, diesel_pi = simulate_policy_trajectory(
    pi_pi, T, initial_state, seed=123)
428 plot_policy_trajectory(
429     'Policy Iteration: B_t, L_t, G_t, a_t y rewards',
430     B_pi,
431     L_pi,
432     G_pi,
433     a_pi,
434     Rcum_pi,
435     Ravg_pi,
436     save_path=os.path.join(IMG_DIR, 'policy_iteration_trajectory.png')
437 )
438
439 # Gráfico Value Iteration
440 B_vi, L_vi, G_vi, a_vi, Rcum_vi, Ravg_vi, r_vi, diesel_vi = simulate_policy_trajectory(
    pi_vi, T, initial_state, seed=123)
441 plot_policy_trajectory(
442     'Value Iteration: B_t, L_t, G_t, a_t y rewards',
443     B_vi,
444     L_vi,
445     G_vi,
446     a_vi,
447     Rcum_vi,
448     Ravg_vi,
449     save_path=os.path.join(IMG_DIR, 'value_iteration_trajectory.png')
450 )

```

```

451
452 print('\nResumen trayectoria (Policy Iteration):')
453 print(f'Reward promedio: {np.mean(r_pi):.3f}')
454 print(f'Diesel total usado: {np.sum(diesel_pi):.3f}')
455 print(f'Uso total bater a (|a_t|): {np.sum(np.abs(a_pi)):.3f}')
456
457 print('\nResumen trayectoria (Value Iteration):')
458 print(f'Reward promedio: {np.mean(r_vi):.3f}')
459 print(f'Diesel total usado: {np.sum(diesel_vi):.3f}')
460 print(f'Uso total bater a (|a_t|): {np.sum(np.abs(a_vi)):.3f}')
461
462 # Políticas de VI y PI separadas por momento del día (D_t = m, a, n)
463 plot_policy_action_heatmaps_by_day(
464     pi_pi,
465     pi_vi,
466     V_pi,
467     V_vi,
468     'Policy Iteration (estacionaria)',
469     'Value Iteration (estacionaria)',
470     t_right=0,
471     save_dir=IMG_DIR
472 )

```

Anexos Pregunta 2

0.0.2. p2.py

```

1 from TwinRotorODE import TwinRotor
2 import numpy as np
3 import matplotlib.pyplot as plt
4 from KF import DiscreteKalmanFilter
5
6
7 params = {
8     'Jp': 0.038,
9     'Jy': 0.043,
10    'mgl': 0.32,
11    'Kpp': 0.204,
12    'Kyp': 0.051,
13    'Kpt': 0.011,
14    'Kyt': 0.072,
15    'Bp': 0.05,
16    'By': 0.03,
17    'Tm': 0.11,
18    'Tt': 0.10,
19    'JrOmega': 0.02,
20 }
21
22 Ts = 0.01
23
24 system = TwinRotor(params, Ts)
25
26 reference = [0, np.pi/4]
27 x_eq, u_eq = system.get_eq_point(x0=reference[0], x1=reference[1])
28 system.x = x_eq
29 y_eq = np.array([x_eq[0], x_eq[1]]).reshape(-1, 1)
30

```

```

31 Ad, Bd, Cd, Dd, dt = system.get_linear_system(x_eq, u_eq)
32
33 Q = np.diag([1e-4, 1e-4, 1e-2, 1e-2, 1e-4, 1e-4])
34 R = np.eye(Cd.shape[0]) * 0.001 ** 2
35 P0 = np.eye(Ad.shape[0]) * 10
36 kf = DiscreteKalmanFilter.create(Ad, Bd, Cd, Q, R, P_init=P0, D=Dd, x_init=np.zeros((6,
    1)))
37
38
39 # sim
40 u_hover = [u_eq[0] - 0.01, u_eq[1]] # Small perturbation
41
42 N = 2000 # Steps
43 states = np.zeros((N, 6))
44 states_discrete = np.zeros((N, 6))
45 states_estimated = np.zeros((N, 6))
46 measurements = np.zeros((N, 2))
47 for k in range(N):
48     states[k, :], measurements[k, :] = system.sim(u=u_hover)
49     states_discrete[k, :] = system.sim_discrete(u_hover, Ad, Bd, Cd, Dd, x_eq, u_eq)
50
51     x_hat = kf.step(u=np.array(u_hover).reshape(-1, 1) - np.array(u_eq).reshape(-1, 1),
    y=np.array(measurements[k, :]).reshape(-1, 1) - y_eq.reshape(-1, 1))["x_filt"]
52     states_estimated[k, :] = (x_hat + np.array(x_eq).reshape(-1, 1)).squeeze() #
    filtered state estimate at time k
53
54
55 # plot
56 t_vec = np.arange(N) * Ts
57 labels = [ latex afecta esto]
58
59 fig, axes = plt.subplots(2, 3, figsize=(18, 8))
60 fig.suptitle('TwinRotor Deviation from equilibrium',
61             fontsize=13)
62 for i, ax in enumerate(axes.flat):
63     ax.plot(t_vec, states[:, i], linewidth=1.2, label='NonLinear')
64     ax.plot(t_vec, states_discrete[:, i], linewidth=1.2, label='Linear')
65     ax.plot(t_vec, states_estimated[:, i], linewidth=1.2, label='KF est')
66     ax.set_ylabel(labels[i])
67     ax.set_xlabel('t[s]')
68     ax.grid(True, alpha=0.3)
69     ax.legend()
70 plt.tight_layout()
71
72 fig, ax = plt.subplots(1, 1, figsize=(18, 8))
73 ax.plot(t_vec, measurements, linewidth=1.2, label='Measurements')
74 ax.set_ylabel(labels[i])
75 ax.set_xlabel('t[s]')
76 ax.grid(True, alpha=0.3)
77 ax.legend()
78 plt.tight_layout()
79 plt.show()

```

0.0.3. p2_closed.py

```

1 from TwinRotorODE import TwinRotor, TwinRotorEnhanced
2 from KF import DiscreteKalmanFilter

```

```

3 import numpy as np
4 import control
5 import matplotlib.pyplot as plt
6 import os
7
8
9 params = {
10     "Jp": 0.038,
11     "Jy": 0.043,
12     "mgl": 0.32,
13     "Kpp": 0.204,
14     "Kyp": 0.051,
15     "Kpt": 0.011,
16     "Kyt": 0.072,
17     "Bp": 0.05,
18     "By": 0.03,
19     "Tm": 0.11,
20     "Tt": 0.10,
21     "JrOmega": 0.02,
22 }
23
24 Ts = 0.01
25 N = 2000
26
27 # LQR weights
28 Q_lqr = np.diag([80.0, 80.0, 2.0, 2.0, 0.1, 0.1])
29 R_lqr = np.diag([0.4, 0.4])
30
31 # KF weights
32 Q_kf = np.diag([5e-5, 5e-5, 5e-3, 5e-3, 5e-4, 5e-4])
33 R_kf = np.diag([0.01 ** 2, 0.005 ** 2])
34 P0 = np.eye(6) * 5.0
35
36
37 def simulate_reference(reference):
38     system = TwinRotor(
39         params,
40         Ts,
41         disturbance_time=[1000, 1500],
42         disturbance_strength=0.1,
43     )
44
45     x_eq, u_eq = system.get_eq_point(x0=reference[0], x1=reference[1])
46     x_eq = np.array(x_eq, dtype=float).reshape(-1, 1)
47     u_eq = np.array(u_eq, dtype=float).reshape(-1, 1)
48     y_eq = x_eq[:, 2, :]
49
50     system.x = x_eq.squeeze().tolist()
51
52     # matrices linealizadas del eq (re linealizar en pi = 0 y pi/4)
53     Ad, Bd, Cd, Dd, _ = system.get_linear_system(x_eq.squeeze().tolist(), u_eq.squeeze()
54         .tolist())
55
56     # LQR desde control.dlqr
57     K, _, E = control.dlqr(Ad, Bd, Q_lqr, R_lqr)
58     K = np.asarray(K, dtype=float)
59
60     # kalman
61     kf = DiscreteKalmanFilter.create(

```



```

61     Ad,
62     Bd,
63     Cd,
64     Q_kf,
65     R_kf,
66     P_init=P0,
67     D=Dd,
68     x_init=np.zeros((6, 1)),
69 )
70
71 states = np.zeros((N, 6))
72 states_est = np.zeros((N, 6))
73 measurements = np.zeros((N, 2))
74 controls = np.zeros((N, 2))
75
76 u_cmd = u_eq.copy() # comando init
77
78 # simular
79 for k in range(N):
80     x, y = system.sim(u=u_cmd.squeeze().tolist())
81     x = np.array(x, dtype=float).reshape(-1, 1)
82     y = np.array(y, dtype=float).reshape(-1, 1)
83
84     states[k, :] = x.squeeze()
85     measurements[k, :] = y.squeeze()
86     controls[k, :] = u_cmd.squeeze()
87
88     y_dev = y - y_eq
89     u_dev = u_cmd - u_eq
90
91     x_hat_dev = kf.step(u=u_dev, y=y_dev)["x_filt"]
92     x_hat = x_hat_dev + x_eq
93     states_est[k, :] = x_hat.squeeze()
94
95     u_dev_next = -K @ x_hat_dev
96     u_cmd = u_eq + u_dev_next
97     u_cmd = np.clip(u_cmd, -5.0, 5.0)
98
99 # medir errores
100 err = np.abs(states[:, :2] - y_eq.squeeze())
101 iae_phi = np.sum(err[:, 0]) * Ts
102 iae_psi = np.sum(err[:, 1]) * Ts
103 iae_total = iae_phi + iae_psi
104
105 i0, i1 = [1000, 1500]
106 iae_dist_phi = np.sum(err[i0:i1, 0]) * Ts # iae phi disturbance
107 iae_dist_psi = np.sum(err[i0:i1, 1]) * Ts # iae psi disturbance
108 iae_dist_total = iae_dist_phi + iae_dist_psi # total iae disturbance
109
110 metrics = {
111     "iae_phi": iae_phi,
112     "iae_psi": iae_psi,
113     "iae_total": iae_total,
114     "iae_dist_phi": iae_dist_phi,
115     "iae_dist_psi": iae_dist_psi,
116     "iae_dist_total": iae_dist_total,
117 }
118
119 return {

```

```

120         "ref": reference,
121         "x_eq": x_eq.squeeze(),
122         "u_eq": u_eq.squeeze(),
123         "K": K,
124         "states": states,
125         "states_est": states_est,
126         "measurements": measurements,
127         "controls": controls,
128         "metrics": metrics,
129     }
130
131
132 def main():
133     references = [
134         (0.0, 0.0),
135         (0.0, np.pi / 4),
136         (np.pi / 4, np.pi / 4),
137     ]
138
139     results = [simulate_reference(ref) for ref in references]
140
141     t_vec = np.arange(N) * Ts
142     base_dir = os.path.dirname(os.path.abspath(__file__))
143     results_dir = os.path.join(base_dir, "results")
144     os.makedirs(results_dir, exist_ok=True)
145
146     fig, axes = plt.subplots(3, 2, figsize=(14, 12), sharex=True)
147
148     # plt
149     for idx, res in enumerate(results):
150         r0, r1 = res["ref"]
151         states = res["states"]
152         states_est = res["states_est"]
153         controls = res["controls"]
154
155         ax_ang = axes[idx, 0]
156         ax_u = axes[idx, 1]
157
158         ax_ang.plot(t_vec, states[:, 0], label="phi␣(nonlinear)", lw=1.1)
159         ax_ang.plot(t_vec, states[:, 1], label="psi␣(nonlinear)", lw=1.1)
160         ax_ang.plot(t_vec, states_est[:, 0], "--", label="phi_hat␣(KF)", lw=1.0)
161         ax_ang.plot(t_vec, states_est[:, 1], "--", label="psi_hat␣(KF)", lw=1.0)
162         ax_ang.axhline(r0, color="k", ls=":", lw=1.0, label="phi_ref" if idx == 0 else
163             None)
164         ax_ang.axhline(r1, color="gray", ls=":", lw=1.0, label="psi_ref" if idx == 0
165             else None)
166         ax_ang.axvspan(1000 * Ts, 1500 * Ts, color="red", alpha=0.08)
167         ax_ang.set_ylabel("angles␣[rad]")
168         ax_ang.set_title(f"Ref=({r0:.3f},␣{r1:.3f})␣|␣IAE={res['metrics']['iae_total']:.3f}")
169         ax_ang.grid(True, alpha=0.3)
170         ax_ang.legend(loc="upper␣right", ncol=2, fontsize=8)
171
172         ax_u.plot(t_vec, controls[:, 0], label="u0", lw=1.1)
173         ax_u.plot(t_vec, controls[:, 1], label="u1", lw=1.1)
174         ax_u.axvspan(1000 * Ts, 1500 * Ts, color="red", alpha=0.08)
175         ax_u.set_ylabel("input␣[V]")
176         ax_u.grid(True, alpha=0.3)
177         ax_u.legend(loc="upper␣right", fontsize=8)

```

```

176
177     axes[-1, 0].set_xlabel("time[s]")
178     axes[-1, 1].set_xlabel("time[s]")
179     fig.suptitle("Closed-loop de-twin rotor control LQR Kalman Filter", fontsize=16)
180     fig.tight_layout()
181
182     fig_path = os.path.join(results_dir, "closed_loop_kf_lqr.png")
183     fig.savefig(fig_path, dpi=150)
184
185     print("=== RESUMEN ===")
186     for i, res in enumerate(results, start=1):
187         ref = res["ref"]
188         m = res["metrics"]
189         print(
190             f"Case {i} ref=({ref[0]:.5f}, {ref[1]:.5f}) | "
191             f"IAE_phi={m['iae_phi']:.4f}, IAE_psi={m['iae_psi']:.4f}, IAE_total={m['iae_total']:.4f}, "
192             f"IAE_dist={m['iae_dist_total']:.4f}"
193         )
194
195 if __name__ == "__main__":
196     main()

```

0.0.4. p2_open.py

```

1 from TwinRotorODE import TwinRotor
2 import numpy as np
3 import matplotlib
4 matplotlib.use("Agg")
5 import matplotlib.pyplot as plt
6 import os
7
8 params = {
9     "Jp": 0.038,
10    "Jy": 0.043,
11    "mgl": 0.32,
12    "Kpp": 0.204,
13    "Kyp": 0.051,
14    "Kpt": 0.011,
15    "Kyt": 0.072,
16    "Bp": 0.05,
17    "By": 0.03,
18    "Tm": 0.11,
19    "Tt": 0.10,
20    "JrOmega": 0.02,
21 }
22
23 Ts = 0.01
24 N = 2000
25 step_amp = 0.05
26 base_dir = os.path.dirname(os.path.abspath(__file__))
27 results_dir = os.path.join(base_dir, "results")
28 os.makedirs(results_dir, exist_ok=True)
29
30
31 def run_case(u_step_idx, step_amp_value):
32     system = TwinRotor(params, Ts)

```

```

33     # Equilibrio    x0 = 0, x1 = 0
34     x_eq, u_eq = system.get_eq_point(x0=0, x1=0)
35     system.x = x_eq
36     system.t = 0
37
38     u = np.array(u_eq, dtype=float)
39     u[u_step_idx] += step_amp_value
40
41     states = np.zeros((N, 6))
42     outputs = np.zeros((N, 2))
43
44     for k in range(N):
45         x, y = system.sim(u=u.tolist())
46         states[k, :] = np.array(x, dtype=float)
47         outputs[k, :] = np.array(y, dtype=float)
48
49     return np.array(x_eq, dtype=float), np.array(u_eq, dtype=float), u, states, outputs
50
51
52 x_eq_1, u_eq_1, u_1, states_1, outputs_1 = run_case(u_step_idx=0, step_amp_value=
    step_amp)
53 x_eq_2, u_eq_2, u_2, states_2, outputs_2 = run_case(u_step_idx=1, step_amp_value=
    step_amp)
54
55 t_vec = np.arange(N) * Ts
56
57 fig, axes = plt.subplots(2, 2, figsize=(12, 7), sharex=True)
58
59 axes[0, 0].plot(t_vec, states_1[:, 0], label="phi")
60 axes[0, 0].plot(t_vec, states_1[:, 1], label="psi")
61 axes[0, 0].set_title("Step in pitch input (u0)")
62 axes[0, 0].set_ylabel("angles [rad]")
63 axes[0, 0].grid(True, alpha=0.3)
64 axes[0, 0].legend()
65
66 axes[1, 0].plot(t_vec, np.full_like(t_vec, u_1[0]), label="u0")
67 axes[1, 0].plot(t_vec, np.full_like(t_vec, u_1[1]), label="u1")
68 axes[1, 0].set_ylabel("inputs [V]")
69 axes[1, 0].set_xlabel("time [s]")
70 axes[1, 0].grid(True, alpha=0.3)
71 axes[1, 0].legend()
72
73 axes[0, 1].plot(t_vec, states_2[:, 0], label="phi")
74 axes[0, 1].plot(t_vec, states_2[:, 1], label="psi")
75 axes[0, 1].set_title("Step in yaw input (u1)")
76 axes[0, 1].set_ylabel("angles [rad]")
77 axes[0, 1].grid(True, alpha=0.3)
78 axes[0, 1].legend()
79
80 axes[1, 1].plot(t_vec, np.full_like(t_vec, u_2[0]), label="u0")
81 axes[1, 1].plot(t_vec, np.full_like(t_vec, u_2[1]), label="u1")
82 axes[1, 1].set_ylabel("inputs [V]")
83 axes[1, 1].set_xlabel("time [s]")
84 axes[1, 1].grid(True, alpha=0.3)
85 axes[1, 1].legend()
86
87 fig.suptitle("Twin Rotor open-loop response from equilibrium x0=0, x1=0")
88 fig.tight_layout()
89 fig.savefig(os.path.join(results_dir, "open_loop_steps.png"), dpi=150)

```

```

90
91 phi_change_u0 = np.mean(states_1[-200:, 0] - x_eq_1[0])
92 psi_change_u0 = np.mean(states_1[-200:, 1] - x_eq_1[1])
93 phi_change_u1 = np.mean(states_2[-200:, 0] - x_eq_2[0])
94 psi_change_u1 = np.mean(states_2[-200:, 1] - x_eq_2[1])

```

0.0.5. p2_lqr_dp.py

```

1  from TwinRotorODE import TwinRotor
2  import numpy as np
3  import control
4
5
6  params = {
7      'Jp': 0.038,
8      'Jy': 0.043,
9      'mgl': 0.32,
10     'Kpp': 0.204,
11     'Kyp': 0.051,
12     'Kpt': 0.011,
13     'Kyt': 0.072,
14     'Bp': 0.05,
15     'By': 0.03,
16     'Tm': 0.11,
17     'Tt': 0.10,
18     'JrOmega': 0.02,
19 }
20
21 Ts = 0.01
22 system = TwinRotor(params, Ts)
23
24 # modelo linealizado para el eq
25 x_eq, u_eq = system.get_eq_point(x0=0, x1=np.pi / 4)
26 Ad, Bd, Cd, Dd, dt = system.get_linear_system(x_eq, u_eq)
27
28 # Matrices de costo
29 Q = np.diag([1e-4, 1e-4, 1e-2, 1e-2, 1e-4, 1e-4])
30 R = np.eye(Bd.shape[1]) * 0.001 ** 2
31
32
33 # ricatti dp
34 def riccati_dp(A, B, Q, R, max_iter=20000, tol=1e-7):
35     P = Q.copy()
36     K = None
37     history = []
38
39     for k in range(max_iter):
40         S = R + B.T @ P @ B
41         K = np.linalg.solve(S, B.T @ P @ A)
42         P_next = Q + A.T @ P @ A - A.T @ P @ B @ K
43         P_next = 0.5 * (P_next + P_next.T)
44
45         diff = np.linalg.norm(P_next - P, ord='fro')
46         history.append(diff)
47         P = P_next
48
49         if diff < tol:

```

```

50         return K, P, k + 1, np.array(history), True
51
52     return K, P, max_iter, np.array(history), False
53
54
55 K_dp, P_dp, iters, history, converged = riccati_dp(Ad, Bd, Q, R)
56 K_dlqr, P_dlqr, E_dlqr = control.dlqr(Ad, Bd, Q, R)
57 K_dlqr = np.asarray(K_dlqr)
58 P_dlqr = np.asarray(P_dlqr)
59 E_dlqr = np.asarray(E_dlqr)
60
61 err_abs = np.linalg.norm(K_dp - K_dlqr, ord='fro')
62 err_rel = err_abs / max(np.linalg.norm(K_dlqr, ord='fro'), 1e-16)
63
64 # imprimir resultados
65
66 np.set_printoptions(precision=8, suppress=True)
67 print("== Discrete LQR via dynamic programming ==")
68 print(f"Converged: {converged}")
69 print(f"Iterations until convergence: {iters}")
70 print(f"Final Riccati delta: {history[-1]:.3e}")
71 print("K from dynamic programming:")
72 print(K_dp)
73 print("\nK from control.dlqr:")
74 print(K_dlqr)
75 print("\n||K_dp - K_dlqr||:")
76 print(err_abs)
77 print("Relative difference:")
78 print(err_rel)
79 print("\nClosed-loop eigenvalues from control.dlqr:")
80 print(E_dlqr)

```

0.0.6. p2_NCS.py

```

1  import os
2
3  import numpy as np
4  import matplotlib.pyplot as plt
5  from scipy.stats import beta as beta_dist
6
7  from channel_selector import ChannelSelector
8
9
10 def thompson_sampling_simulation(n_steps=2500, n_channels=6, seed=0, ts_comm=0.05):
11     # selector de canales
12     channel_selector = ChannelSelector(n_channels=n_channels, seed=seed)
13
14     alpha = np.ones(n_channels) # alpha inicial = 1 por canal
15     beta_params = np.ones(n_channels) # beta tb
16
17     chosen_channels = np.zeros(n_steps, dtype=int) # canal de cada paso
18     # diccionario de veces elegidas por canal
19     chosen_channels_dict = {i: [] for i in range(n_channels)}
20
21     rewards = np.zeros(n_steps)
22     cumulative_reward = np.zeros(n_steps)
23     cumulative_regret = np.zeros(n_steps)

```

```

24     estimated_means = np.zeros((n_steps, n_channels))
25     true_probs = channel_selector.probs.copy()
26     best_prob = float(np.max(true_probs))
27     rng = np.random.default_rng(seed)
28
29     for k in range(n_steps):
30         # clase MAB Thompson Sampling
31         # beta sampling
32         samples = rng.beta(alpha, beta_params)
33
34         # elegimos el canal con mayor valor
35         channel = int(np.argmax(samples))
36         chosen_channels[k] = channel
37         chosen_channels_dict[channel].append(k)
38
39         # apply action y reward
40         success = channel_selector.send_u(np.zeros((2, 1)), channel=channel)
41         reward = 1.0 if success else 0.0
42
43         rewards[k] = reward
44         cumulative_reward[k] = rewards[:k + 1].sum()
45         cumulative_regret[k] = (cumulative_regret[k - 1] if k > 0 else 0.0) + (best_prob
46                                     - true_probs[channel])
47
48         # actualizar distribuciones
49         if reward > 0:
50             alpha[channel] += 1.0
51         else:
52             beta_params[channel] += 1.0
53
54         estimated_means[k, :] = alpha / (alpha + beta_params)
55
56     return {
57         "ts": ts_comm,
58         "true_probs": true_probs,
59         "best_prob": best_prob,
60         "chosen_channels": chosen_channels,
61         "rewards": rewards,
62         "cumulative_reward": cumulative_reward,
63         "cumulative_regret": cumulative_regret,
64         "estimated_means": estimated_means,
65         "alpha": alpha,
66         "beta": beta_params,
67         "chosen_channels_dict": chosen_channels_dict,
68     }
69
70 def epsilon_greedy_simulation(true_probs, n_steps=2500, epsilon=0.1, seed=1, ts_comm
71                               =0.05):
72     # igual pero sin alpha beta y distribuciones
73     n_channels = len(true_probs)
74     channel_selector = ChannelSelector(n_channels=n_channels, seed=seed)
75     channel_selector.probs = np.array(true_probs, dtype=float)
76
77     counts = np.zeros(n_channels, dtype=int)
78     means = np.zeros(n_channels)
79
80     chosen_channels = np.zeros(n_steps, dtype=int)
81     chosen_channels_dict = {i: [] for i in range(n_channels)}

```

```

81     rewards = np.zeros(n_steps)
82     cumulative_reward = np.zeros(n_steps)
83     cumulative_regret = np.zeros(n_steps)
84     best_prob = float(np.max(true_probs))
85     rng = np.random.default_rng(seed)
86
87     for k in range(n_steps):
88         explore = rng.random() < epsilon
89         if explore or np.all(counts == 0):
90             channel = int(rng.integers(0, n_channels))
91         else:
92             channel = int(np.argmax(means))
93
94         chosen_channels[k] = channel
95         chosen_channels_dict[channel].append(k)
96
97         success = channel_selector.send_u(np.zeros((2, 1)), channel=channel)
98         reward = 1.0 if success else 0.0
99
100        rewards[k] = reward
101        cumulative_reward[k] = rewards[:k + 1].sum()
102        cumulative_regret[k] = (cumulative_regret[k - 1] if k > 0 else 0.0) + (best_prob
            - true_probs[channel])
103
104        counts[channel] += 1
105        means[channel] += (reward - means[channel]) / counts[channel]
106
107    return {
108        "ts": ts_comm,
109        "true_probs": np.array(true_probs, dtype=float),
110        "best_prob": best_prob,
111        "chosen_channels": chosen_channels,
112        "rewards": rewards,
113        "cumulative_reward": cumulative_reward,
114        "cumulative_regret": cumulative_regret,
115        "means": means,
116        "counts": counts,
117        "epsilon": epsilon,
118        "chosen_channels_dict": chosen_channels_dict,
119    }
120
121 if __name__ == "__main__":
122     base_dir = os.path.dirname(os.path.abspath(__file__))
123     results_dir = os.path.join(base_dir, "results")
124     os.makedirs(results_dir, exist_ok=True)
125
126     # params
127     n_steps=2500
128     n_channels=6
129     seed=0
130
131     # sampling
132     ts_comm=0.05
133
134     # epsilon greedy
135     epsilon=0.1
136
137     ## THOMPSON SAMPLING

```



```

138 comm = thompson_sampling_simulation(n_steps=n_steps, n_channels=n_channels, seed=
    seed, ts_comm=ts_comm)
139 t_comm = np.arange(len(comm["rewards"])) * comm["ts"]
140 best_channel = int(np.argmax(comm["true_probs"]))
141
142 fig2, axes2 = plt.subplots(2, 1, figsize=(12, 8), sharex=True)
143 axes2[0].plot(t_comm, comm["cumulative_reward"], lw=1.4)
144 axes2[0].set_ylabel("cumulative_reward")
145 axes2[0].grid(True, alpha=0.3)
146 axes2[0].set_title("Thompson Sampling for wireless channel selection")
147
148 axes2[1].step(t_comm, comm["chosen_channels"], where="post", lw=1.0)
149 axes2[1].axhline(best_channel, color="k", ls=":", lw=1.0, label=f"best_channel={
    best_channel}")
150 axes2[1].set_ylabel("selected_channel")
151 axes2[1].set_xlabel(f"time[s] (Ts={comm['ts']})")
152 axes2[1].grid(True, alpha=0.3)
153 axes2[1].legend()
154
155 fig2.tight_layout()
156 fig2_path = os.path.join(results_dir, "thompson_sampling_channel_reward.png")
157 fig2.savefig(fig2_path, dpi=150)
158
159 x_beta = np.linspace(0.0, 1.0, 400)
160 fig3, ax3 = plt.subplots(1, 1, figsize=(13, 7))
161 colors = plt.cm.tab10(np.linspace(0, 1, len(comm["alpha"])))
162 for i in range(len(comm["alpha"])):
163     a_i = comm["alpha"][i]
164     b_i = comm["beta"][i]
165     y_beta = beta_dist.pdf(x_beta, a_i, b_i)
166     mean_i = a_i / (a_i + b_i)
167     y_mean = beta_dist.pdf(mean_i, a_i, b_i)
168     n_selected = len(comm["chosen_channels_dict"][i])
169
170     ax3.plot(x_beta, y_beta, lw=1.8, color=colors[i], label=f"Ch_{i}")
171     ax3.axvline(comm["true_probs"][i], color=colors[i], ls=":", lw=1.0, alpha=0.8)
172     ax3.scatter([mean_i], [y_mean], color=colors[i], s=22, zorder=3)
173     ax3.text(mean_i, y_mean, f"n={n_selected}", color=colors[i], fontsize=9, va="
        bottom")
174
175 ax3.set_title("Posterior Beta distributions (all channels)")
176 ax3.set_xlabel("p")
177 ax3.set_ylabel("pdf")
178 ax3.grid(True, alpha=0.3)
179 ax3.legend(ncol=3, fontsize=9)
180 fig3.tight_layout()
181 fig3_path = os.path.join(results_dir, "thompson_beta_posteriors.png")
182 fig3.savefig(fig3_path, dpi=150)
183
184 ## EPLSILON GREEDY
185 eps_comm = epsilon_greedy_simulation(comm["true_probs"], n_steps=n_steps, epsilon=
    epsilon, seed=1, ts_comm=ts_comm)
186
187 fig_eps, axes_eps = plt.subplots(2, 1, figsize=(12, 8), sharex=True)
188 axes_eps[0].plot(t_comm, eps_comm["cumulative_reward"], lw=1.4)
189 axes_eps[0].set_ylabel("cumulative_reward")
190 axes_eps[0].grid(True, alpha=0.3)
191 axes_eps[0].set_title(f"Epsilon-greedy for wireless channel selection (eps={epsilon
    })")

```

```

192 axes_eps[1].step(t_comm, eps_comm["chosen_channels"], where="post", lw=1.0)
193 axes_eps[1].axhline(best_channel, color="k", ls=":", lw=1.0, label=f"best_channel={best_channel}")
194 axes_eps[1].set_ylabel("selected_channel")
195 axes_eps[1].set_xlabel(f"time[s](Ts={comm['ts']})")
196 axes_eps[1].grid(True, alpha=0.3)
197 axes_eps[1].legend()
198
199
200 fig_eps.tight_layout()
201 fig_eps_path = os.path.join(results_dir, "epsilon_greedy_channel_reward.png")
202 fig_eps.savefig(fig_eps_path, dpi=150)
203
204 fig4, ax4 = plt.subplots(1, 1, figsize=(12, 6))
205 ax4.plot(t_comm, comm["cumulative_regret"], label="Thompson Sampling", lw=1.6)
206 ax4.plot(t_comm, eps_comm["cumulative_regret"], label=f"epsilon-greedy(eps={epsilon})", lw=1.6)
207 ax4.set_xlabel(f"time[s](Ts={comm['ts']})")
208 ax4.set_ylabel("cumulative_regret")
209 ax4.grid(True, alpha=0.3)
210 ax4.legend()
211 ax4.set_title("Cumulative_regret_comparison")
212 fig4.tight_layout()
213 fig4_path = os.path.join(results_dir, "regret_thompson_vs_epsilon_greedy.png")
214 fig4.savefig(fig4_path, dpi=150)
215
216 print("=== THOMPSON SAMPLING / CHANNEL SELECTION ===")
217 print(f"True channel reliabilities: {comm['true_probs']}")
218 print(f"Best true channel: {best_channel}")
219 print(f"Final cumulative regret: {comm['cumulative_regret'][-1]:.4f}")
220 print(f"Final cumulative reward: {comm['cumulative_reward'][-1]:.2f}")
221
222 print("=== EPSILON-GREEDY COMPARISON ===")
223 print(f"Final cumulative regret (epsilon-greedy): {eps_comm['cumulative_regret'][-1]:.4f}")
224 print(f"Final cumulative reward (epsilon-greedy): {eps_comm['cumulative_reward'][-1]:.2f}")

```

Anexos Pregunta 3

0.0.7. p3.py

```

1 from evacuation import EvacuationEnv
2 from collections import defaultdict
3 import random
4 import numpy as np
5 import matplotlib.pyplot as plt
6 import os
7
8 results_dir = os.path.join(os.path.dirname(os.path.abspath(__file__)), "results")
9 os.makedirs(results_dir, exist_ok=True)
10
11
12 edges = [
13     ("A", "B", 2.6, 3, 0), ("B", "C", 2.4, 2, 0), ("C", "D", 2.2, 2, 0), ("D", "S1",
14     2.0, 2, 0),
15     ("E", "F", 2.8, 4, 0), ("F", "G", 2.8, 4, 0), ("G", "H", 2.8, 4, 0), ("H", "S2",
16     2.5, 3, 0),

```

```

15         ("E", "I", 3.2, 5, 0), ("I", "J", 3.2, 5, 0), ("J", "H", 3.2, 5, 0),
16         ("A", "E", 3.0, 2, 0), ("B", "F", 2.6, 2, 0), ("C", "G", 2.6, 2, 0), ("D", "H",
17         3.0, 2, 0),
18         ("B", "E", 2.8, 2, 0), ("C", "F", 2.6, 1, 0), ("C", "H", 3.4, 2, 0),
19         ("F", "I", 2.8, 3, 0), ("G", "J", 2.8, 3, 0), ]
20
21 positions = { # For plotting
22     "A": (0, 2),
23     "B": (2, 2),
24     "C": (4, 2),
25     "D": (6, 2),
26
27     "E": (0, 0),
28     "F": (2, 0),
29     "G": (4, 0),
30     "H": (6, 0),
31
32     "I": (1, -2),
33     "J": (5, -2),
34
35     "S1": (8, 2),
36     "S2": (8, 0),
37 }
38
39
40 ## setup
41 env = EvacuationEnv(n_agents=50, s0=["A", "B", "C", "D", "E", "F", "G", "H", "I", "J"
42     ]*5,
43     edges=edges, node_positions=positions)
44 # print(env.get_neighbors('A'))
45
46 ###----- pol tica epsilon-greedy -----###
47 def epsilon_greedy_policy(q_table, state, valid_actions, epsilon):
48     if not valid_actions:
49         return 0
50
51     max_valid_action = max(valid_actions)
52     if max_valid_action >= len(q_table[state]):
53         old_q = q_table[state]
54         pad = max_valid_action + 1 - len(old_q)
55         q_table[state] = np.pad(old_q, (0, pad), mode="constant")
56
57     if random.random() < epsilon:
58         return random.choice(valid_actions)
59     else:
60         q_values = [q_table[state][action] for action in valid_actions]
61         max_q = max(q_values)
62         best_actions = [action for action in valid_actions if q_table[state][action] ==
63             max_q]
64         return random.choice(best_actions)
65
66 ###----- Q-Learning -----###
67 def train_q_learning(env, n_episodes=1000, max_steps=60,
68     alpha=0.1, gamma=0.95,
69     eps_start=0.2, eps_end=0.01, eps_decay=0.995):
70     q_table = defaultdict(lambda: np.zeros(env.max_degree))
71     rewards_hist, saved_hist = [], []

```

```

71     epsilon = eps_start
72
73     for ep in range(n_episodes):
74         states = env.reset()
75         ep_reward = 0
76
77         for _ in range(max_steps):
78
79             if all(env.done_list):
80                 break
81
82             valid = env.get_valid_actions_for_current_state()
83             # cada agente elige acci n con eplison greedy propio
84             actions = [epsilon_greedy_policy(q_table, state, valid_actions, epsilon) for
85                        state, valid_actions in zip(states, valid)]
86
87             next_states, rewards, _, _, _, done_list, _, _ = env.step(actions)
88             next_valid = env.get_valid_actions_for_current_state()
89
90             for i in range(env.n_agents):
91                 if rewards[i] is None:
92                     continue
93
94                 # calcular target OFF-policy
95                 if done_list[i]:
96                     target = rewards[i]
97                 else:
98                     target = rewards[i] + gamma * max(q_table[next_states[i]][a] for a
99                        in next_valid[i])
100
101                 # update Q-table
102                 q_table[states[i]][actions[i]] += alpha * (target - q_table[states[i]][
103                     actions[i]])
104
105                 ep_reward += rewards[i]
106
107             states = next_states
108
109             rewards_hist.append(ep_reward)
110             saved_hist.append(sum(env.done_list))
111             epsilon = max(epsilon * eps_decay, eps_end)
112
113         return q_table, rewards_hist, saved_hist
114
115
116 ###----- SARSA -----###
117 def train_sarsa(env, n_episodes=1000, max_steps=60,
118                alpha=0.1, gamma=0.95,
119                eps_start=0.2, eps_end=0.01, eps_decay=0.995):
120     q_table = defaultdict(lambda: np.zeros(env.max_degree))
121     rewards_hist, saved_hist = [], []
122     epsilon = eps_start
123
124     for ep in range(n_episodes):
125         states = env.reset()
126
127         valid = env.get_valid_actions_for_current_state()
128         # cada agente elige acci n con eplison greedy propio

```

```

126     actions = [epsilon_greedy_policy(q_table, state, valid_actions, epsilon) for
127                 state, valid_actions in zip(states, valid)]
128
129     ep_reward = 0.0
130
131     for _ in range(max_steps):
132         if all(env.done_list):
133             break
134
135         next_states, rewards, _, _, _, done_list, _, _ = env.step(actions)
136         next_valid = env.get_valid_actions_for_current_state()
137
138         # on policy
139         # next_actions con epsilon greedy para cada agente
140         next_actions = [epsilon_greedy_policy(q_table, next_states[i], next_valid[i]
141                                             ], epsilon) for i in range(env.n_agents)]
142
143         for i in range(env.n_agents):
144             if rewards[i] is None:
145                 continue
146
147             # calcular target on-policy
148             if done_list[i]:
149                 target = rewards[i]
150             else:
151                 next_action = next_actions[i]
152                 target = rewards[i] + gamma * q_table[next_states[i]][next_action] #
153                                     NO MAX!
154
155             # update Q-table
156             q_table[states[i]][actions[i]] += alpha * (target - q_table[states[i]][
157                 actions[i]])
158
159             ep_reward += rewards[i]
160
161             states = next_states
162             actions = next_actions
163
164             rewards_hist.append(ep_reward)
165             saved_hist.append(sum(env.done_list))
166             epsilon = max(epsilon * eps_decay, eps_end)
167
168         return q_table, rewards_hist, saved_hist
169
170 ###----- PLOTS -----###
171 def moving_average(data, window_size=50):
172     return np.convolve(data, np.ones(window_size)/window_size, mode='valid')
173
174 def plot_progress(q_rewards, q_saved, s_rewards, s_saved):
175     # plot rewards
176     fig, axes = plt.subplots(1, 2, figsize=(14, 5))
177     axes[0].plot(q_rewards, label="Q-Learning_Rewards", alpha=0.25)
178     axes[0].plot(moving_average(q_rewards), label="Q-Learning_MA", linewidth=2)
179     axes[0].plot(s_rewards, label="SARSA_Rewards", alpha=0.25)
180     axes[0].plot(moving_average(s_rewards), label="SARSA_MA", linewidth=2)
181     axes[0].set_title("Episodic_Rewards")

```

```

181 axes[0].set_xlabel("Episode")
182 axes[0].set_ylabel("Total_Reward")
183 axes[0].grid(True, alpha=0.3)
184 axes[0].legend()
185
186 # plot saved
187 axes[1].plot(q_saved, label="Q-Learning_Saved", alpha=0.25)
188 axes[1].plot(moving_average(q_saved), label="Q-Learning_Saved_MA", linewidth=2)
189 axes[1].plot(s_saved, label="SARSA_Saved", alpha=0.25)
190 axes[1].plot(moving_average(s_saved), label="SARSA_Saved_MA", linewidth=2)
191 axes[1].set_title("People_Saved")
192 axes[1].set_xlabel("Episode")
193 axes[1].set_ylabel("Number_of_People_Saved")
194 axes[1].grid(True, alpha=0.3)
195 axes[1].legend()
196 fig.tight_layout()
197 plt.show()
198
199 fig.savefig(results_dir + "/evacuation_training_progress.png", dpi=150)
200
201 def compare_algorithms(q_rewards, q_saved, s_rewards, s_saved, window_size=50):
202     # ltima window de recompensa y personas salvadas
203     q_rewards_avg = np.mean(q_rewards[-window_size:])
204     s_rewards_avg = np.mean(s_rewards[-window_size:])
205     q_saved_avg = np.mean(q_saved[-window_size:])
206     s_saved_avg = np.mean(s_saved[-window_size:])
207
208     print("===Q-Learning_vs_SARSA_final===")
209     print(f"Q-Learning: Final_reward={q_rewards_avg:.2f}, People_saved={q_saved_avg:.2f}")
210     print(f"SARSA: Final_reward={s_rewards_avg:.2f}, People_saved={s_saved_avg:.2f}")
211
212
213 def evaluate_policy_over_time(env, q_table, max_time_seconds=300, selected_agents=None):
214     # politica greedy de la Q-table aprendida
215     if selected_agents is None:
216         selected_agents = [0, 3, 6, 9]
217
218     states = env.reset()
219     times = [0]
220     saved_over_time = [sum(env.done_list)]
221
222     trajectories = {i: [states[i][0]] for i in selected_agents}
223
224     while env.global_time < max_time_seconds and not all(env.done_list):
225         valid = env.get_valid_actions_for_current_state()
226         actions = [
227             epsilon_greedy_policy(q_table, state, valid_actions, epsilon=0.0)
228             for state, valid_actions in zip(states, valid)
229         ]
230
231         next_states, _, _, _, global_time, done_list, _, _ = env.step(actions)
232         states = next_states
233
234         times.append(global_time)
235         saved_over_time.append(sum(done_list))
236
237         for i in selected_agents:

```

```

238         trajectories[i].append(states[i][0])
239
240     return times, saved_over_time, trajectories
241
242
243 def plot_saved_vs_time(times, saved_over_time, title_suffix):
244     fig, ax = plt.subplots(1, 1, figsize=(8, 4))
245     ax.plot(times, saved_over_time, marker="o", linewidth=1.5)
246     ax.set_title(f"People saved vs time ({title_suffix})")
247     ax.set_xlabel("Time[s]")
248     ax.set_ylabel("Saved people")
249     ax.grid(True, alpha=0.3)
250     fig.tight_layout()
251     fig.savefig(os.path.join(results_dir, "evacuation_saved_vs_time.png"), dpi=150)
252
253
254 def plot_trajectories(
255     trajectories,
256     edges=None,
257     positions=None,
258     ax=None,
259     title="Agent trajectories",
260     save_path = None,
261 ):
262     if edges is None:
263         edges = globals()["edges"]
264     if positions is None:
265         positions = globals()["positions"]
266
267     if ax is None:
268         fig, ax = plt.subplots(1, 1, figsize=(8, 6))
269     else:
270         fig = ax.figure
271
272     max_capacity = max(capacity for _, _, _, capacity, _ in edges) # líneas m s anchas
273     # com m s cap
274     for u, v, _, capacity, _ in edges:
275         x1, y1 = positions[u]
276         x2, y2 = positions[v]
277         width = 1.0 + 5.0 * (capacity / max_capacity)
278         ax.plot([x1, x2], [y1, y2], color="0.75", linewidth=width, alpha=0.65, zorder=0)
279
280         # tag
281         mx, my = (x1 + x2) / 2, (y1 + y2) / 2
282         ax.text(mx, my + 0.08, f"C={capacity}", fontsize=7, color="0.35", ha="center",
283               zorder=1)
284
285     # nodos base
286     for node, (x, y) in positions.items():
287         color = "lightgreen" if node.startswith("S") else "lightblue"
288         ax.scatter(x, y, s=300, c=color, edgecolors="k")
289         ax.text(x, y + 0.18, node, ha="center", fontsize=9)
290
291     # trayectorias de agentes seleccionados
292     selected_ids = list(trajectories.keys())
293     colors = plt.cm.tab10(np.linspace(0, 1, max(len(selected_ids), 1)))
294     offset_scale = 0.09 # offset
295
296     for idx, (agent_id, path_nodes) in enumerate(trajectories.items()):

```

```

295     xy = np.array([positions[n] for n in path_nodes], dtype=float)
296
297     # offset
298     dx = ((idx % 5) - 2) * offset_scale
299     dy = ((idx // 5) - 0.5) * offset_scale
300     xy_plot = xy + np.array([dx, dy])
301     color = colors[idx % len(colors)]
302
303     ax.plot(
304         xy_plot[:, 0],
305         xy_plot[:, 1],
306         color=color,
307         linewidth=1.8,
308         alpha=0.9,
309         label=f"Agent_{agent_id}",
310         zorder=3,
311     )
312     ax.scatter(
313         xy_plot[0, 0],
314         xy_plot[0, 1],
315         marker="s",
316         s=90,
317         color=color,
318         edgecolors="k",
319         zorder=5,
320     )
321     end_marker = "*" if path_nodes[-1].startswith("S") else "D" # dif si lleg o no
322     end_size = 180 if end_marker == "*" else 90
323     ax.scatter(
324         xy_plot[-1, 0],
325         xy_plot[-1, 1],
326         marker=end_marker,
327         s=end_size,
328         color=color,
329         edgecolors="k",
330         zorder=6,
331     )
332
333     ax.text(
334         xy_plot[0, 0],
335         xy_plot[0, 1] - 0.17,
336         f"start_{agent_id}",
337         fontsize=7,
338         color=color,
339         ha="center",
340         zorder=7,
341     )
342     ax.text(
343         xy_plot[-1, 0],
344         xy_plot[-1, 1] + 0.17,
345         f"end_{agent_id}",
346         fontsize=7,
347         color=color,
348         ha="center",
349         zorder=7,
350     )
351
352     ax.set_title(title)
353     ax.set_xlabel("x")

```



```

354     ax.set_ylabel("y")
355     ax.grid(True, alpha=0.25)
356     ax.legend()
357     fig.tight_layout()
358     if save_path is not None:
359         fig.savefig(save_path, dpi=150)
360     else:
361         fig.savefig(os.path.join(results_dir, "agent_trajectories.png"), dpi=150)
362
363
364 if __name__ == "__main__":
365     random.seed(42)
366     np.random.seed(42)
367
368     # Q-Learning env
369     env_q = EvacuationEnv(n_agents=50, s0=["A", "B", "C", "D", "E", "F", "G", "H", "I",
370                                           "J"]*5,
371                           edges=edges, node_positions=positions)
372     q_table, q_rewards, q_saved = train_q_learning(env_q, n_episodes=1000)
373
374     # SARSA env
375     env_s = EvacuationEnv(n_agents=50, s0=["A", "B", "C", "D", "E", "F", "G", "H", "I",
376                                           "J"]*5,
377                           edges=edges, node_positions=positions)
378     s_table, s_rewards, s_saved = train_sarsa(env_s, n_episodes=1000)
379
380     plot_progress(q_rewards, q_saved, s_rewards, s_saved)
381     compare_algorithms(q_rewards, q_saved, s_rewards, s_saved, window_size=50)
382
383     # Simulacion entrenada de 300 s
384     env_eval = EvacuationEnv(
385         n_agents=50,
386         s0=["A", "B", "C", "D", "E", "F", "G", "H", "I", "J"] * 5,
387         edges=edges,
388         node_positions=positions,
389     )
390
391     selected_agents = np.random.choice(50, size=10, replace=False) # 10 agentes
392     times, saved_over_time, trajectories = evaluate_policy_over_time(
393         env_eval,
394         q_table, # Q-learning mejor q sars
395         max_time_seconds=300,
396         selected_agents=selected_agents,
397     )
398
399     plot_saved_vs_time(times, saved_over_time, title_suffix="Q-Learning")
400     plot_trajectories(trajectories)

```

0.0.8. p3_salidas.py

0.0.9. p3.py

```

1 import itertools
2 import os
3 import random
4
5 import matplotlib.pyplot as plt

```

```

6 import numpy as np
7
8 from evacuation import EvacuationEnv
9
10 from p3 import train_q_learning, evaluate_policy_over_time
11 from p3 import plot_trajectories
12
13 EvacuationEnv.plot_graph = lambda self, G, safe_nodes=None: None # para q no popupee
    todas las runs
14
15 results_dir = os.path.join(os.path.dirname(os.path.abspath(__file__)), "results")
16 os.makedirs(results_dir, exist_ok=True)
17
18 BASE_EDGES = [
19     ("A", "B", 2.6, 3, 0), ("B", "C", 2.4, 2, 0), ("C", "D", 2.2, 2, 0),
20     ("E", "F", 2.8, 4, 0), ("F", "G", 2.8, 4, 0), ("G", "H", 2.8, 4, 0),
21     ("E", "I", 3.2, 5, 0), ("I", "J", 3.2, 5, 0), ("J", "H", 3.2, 5, 0),
22     ("A", "E", 3.0, 2, 0), ("B", "F", 2.6, 2, 0), ("C", "G", 2.6, 2, 0), ("D", "H", 3.0,
        2, 0),
23     ("B", "E", 2.8, 2, 0), ("C", "F", 2.6, 1, 0), ("C", "H", 3.4, 2, 0),
24     ("F", "I", 2.8, 3, 0), ("G", "J", 2.8, 3, 0),
25 ]
26
27 BASE_POSITIONS = {
28     "A": (0, 2), "B": (2, 2), "C": (4, 2), "D": (6, 2),
29     "E": (0, 0), "F": (2, 0), "G": (4, 0), "H": (6, 0),
30     "I": (1, -2), "J": (5, -2),
31 }
32
33 START_NODES = ["A", "B", "C", "D", "E", "F", "G", "H", "I", "J"]
34 S0 = START_NODES * 5
35
36 # par metros de salidas iguales a base
37 S1_LENGTH, S1_CAPACITY = 2.0, 2
38 S2_LENGTH, S2_CAPACITY = 2.5, 3
39
40 # helper para unir salidas a grafo base
41 def build_config_edges(s1_attach, s2_attach):
42     edges = list(BASE_EDGES)
43     edges.append((s1_attach, "S1", S1_LENGTH, S1_CAPACITY, 0))
44     edges.append((s2_attach, "S2", S2_LENGTH, S2_CAPACITY, 0))
45     return edges
46
47
48 def build_config_positions(s1_attach, s2_attach):
49     positions = dict(BASE_POSITIONS)
50     x1, y1 = positions[s1_attach]
51     x2, y2 = positions[s2_attach]
52
53     # Graficar salidas en nodos attached
54     positions["S1"] = (x1 + 2.0, y1 + 0.4)
55     positions["S2"] = (x2 + 2.0, y2 - 0.4)
56     return positions
57
58 def make_env(s1_attach, s2_attach):
59     return EvacuationEnv(
60         n_agents=50,
61         s0=S0,
62         edges=build_config_edges(s1_attach, s2_attach),

```

```

63         node_positions=build_config_positions(s1_attach, s2_attach),
64     )
65
66 def evaluate_configuration_with_q_learning(s1_attach, s2_attach, n_episodes=200):
67     # Reentrenar Q-learning para el grafo específico
68     env_train = make_env(s1_attach, s2_attach)
69     q_table, q_rewards, q_saved = train_q_learning(env_train, n_episodes=n_episodes)
70
71     env_eval = make_env(s1_attach, s2_attach)
72     times, saved, trajectories = evaluate_policy_over_time(
73         env_eval,
74         q_table,
75         max_time_seconds=300,
76         selected_agents=list(range(10)),
77     )
78
79     return {
80         "config": (s1_attach, s2_attach),
81         "policy": "Q-learning(retrained)",
82         "times": times,
83         "saved": saved,
84         "trajectories": trajectories,
85         "train_reward_last50": float(np.mean(q_rewards[-50:])),
86         "train_saved_last50": float(np.mean(q_saved[-50:])),
87         "score": float(saved[-1]), # rankiar por m s salvados en ltima poca
88     }
89
90
91 def plot_best_vs_original(original_res, best_res):
92     fig, ax = plt.subplots(1, 1, figsize=(9, 5))
93     ax.plot(original_res["times"], original_res["saved"], linewidth=2, label=f"Original_
94         ({original_res['config']},_{original_res['policy']})")
95     ax.plot(best_res["times"], best_res["saved"], linewidth=2, label=f"Best_({best_res['
96         config']},_{best_res['policy']})")
97     ax.set_title("Evacuated_agents_vs_time(max_300s)")
98     ax.set_xlabel("Time[s]")
99     ax.set_ylabel("Evacuated_agents")
100     ax.grid(True, alpha=0.3)
101     ax.legend()
102     fig.tight_layout()
103     fig.savefig(os.path.join(results_dir, "config_best_vs_original.png"), dpi=150)
104
105 def plot_config_trajectory(res, rank_label):
106     cfg = res["config"]
107     s1_attach, s2_attach = cfg
108     cfg_edges = build_config_edges(s1_attach, s2_attach)
109     cfg_positions = build_config_positions(s1_attach, s2_attach)
110     trajectories = res["trajectories"]
111
112     fname = f"trajectory_{rank_label.lower().replace(' ', '_')}_S1_{s1_attach}_S2_{
113         s2_attach}.png"
114     plot_trajectories(
115         trajectories,
116         edges=cfg_edges,
117         positions=cfg_positions,
118         title=f"{rank_label}_S1->{s1_attach}_S2->{s2_attach}|_Final_evacuated={int(
119             res['saved'][-1])}",
120         save_path=os.path.join(results_dir, fname),

```

```

118     )
119
120
121 def plot_saved_over_time_selected(config_results):
122     fig, ax = plt.subplots(1, 1, figsize=(10, 5))
123     for res in config_results:
124         s1, s2 = res["config"]
125         label = f"S1->{s1}, S2->{s2} (final={int(res['saved'][-1])})"
126         ax.plot(res["times"], res["saved"], linewidth=2, label=label)
127
128     ax.set_title("People saved over time (Base+Top5 configs)")
129     ax.set_xlabel("Time [s]")
130     ax.set_ylabel("Evacuated agents")
131     ax.grid(True, alpha=0.3)
132     ax.legend(fontsize=8)
133     fig.tight_layout()
134     fig.savefig(os.path.join(results_dir, "top5_plus_base_saved_over_time.png"), dpi
135                 =150)
136
137 def main():
138     random.seed(42)
139     np.random.seed(42)
140
141     # Base config
142     original_cfg = ("D", "H")
143
144     candidate_nodes = START_NODES
145     all_cfgs = [(a, b) for a, b in itertools.permutations(candidate_nodes, 2)]
146
147     # evaluar todas las configuraciones
148     sampled_cfgs = all_cfgs
149
150     results = []
151     for idx, cfg in enumerate(sampled_cfgs, start=1):
152         print(f"Evaluating config {idx}/{len(sampled_cfgs)}: S1->{cfg[0]}, S2->{cfg[1]}")
153         res = evaluate_configuration_with_q_learning(
154             cfg[0],
155             cfg[1],
156             n_episodes=200,
157         )
158         results.append(res)
159
160     # mejores configuraciones
161     best_res = max(results, key=lambda r: r["score"])
162
163     # meter original para comparar
164     original_res = next((r for r in results if r["config"] == original_cfg), None)
165     if original_res is None:
166         original_res = evaluate_configuration_with_q_learning(
167             original_cfg[0],
168             original_cfg[1],
169             n_episodes=200,
170         )
171
172     plot_best_vs_original(original_res, best_res)
173
174     # Base y top 5 configs
175     ranked = sorted(results, key=lambda r: r["score"], reverse=True)

```

```

175     top5_ex_base = [r for r in ranked if r["config"] != original_cfg][:5] # mejores sin
    la base
176     selected_6 = [original_res] + top5_ex_base
177
178     # trayectorias
179     plot_config_trajectory(original_res, rank_label="Base")
180     for k, res in enumerate(top5_ex_base, start=1):
181         plot_config_trajectory(res, rank_label=f"Top_{k}")
182
183     # saved de todas
184     plot_saved_over_time_selected(selected_6)
185
186 if __name__ == "__main__":
187     main()

```

Referencias